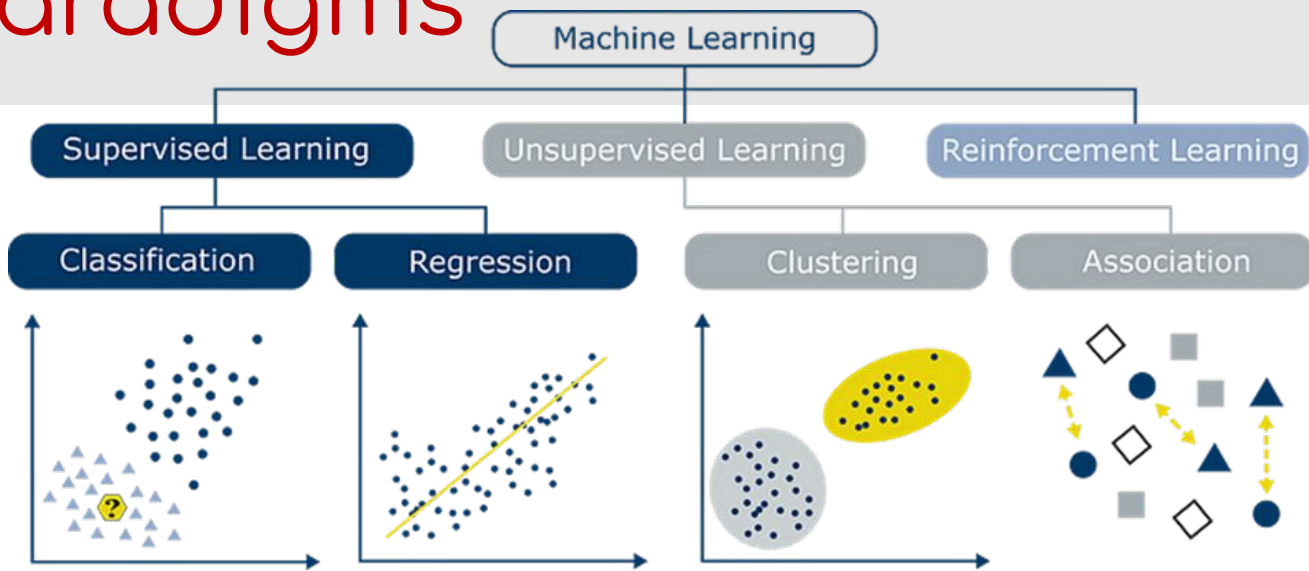


Computer vision

using Deep Neural Networks (DNN)

Basics of machine learning - recap.?

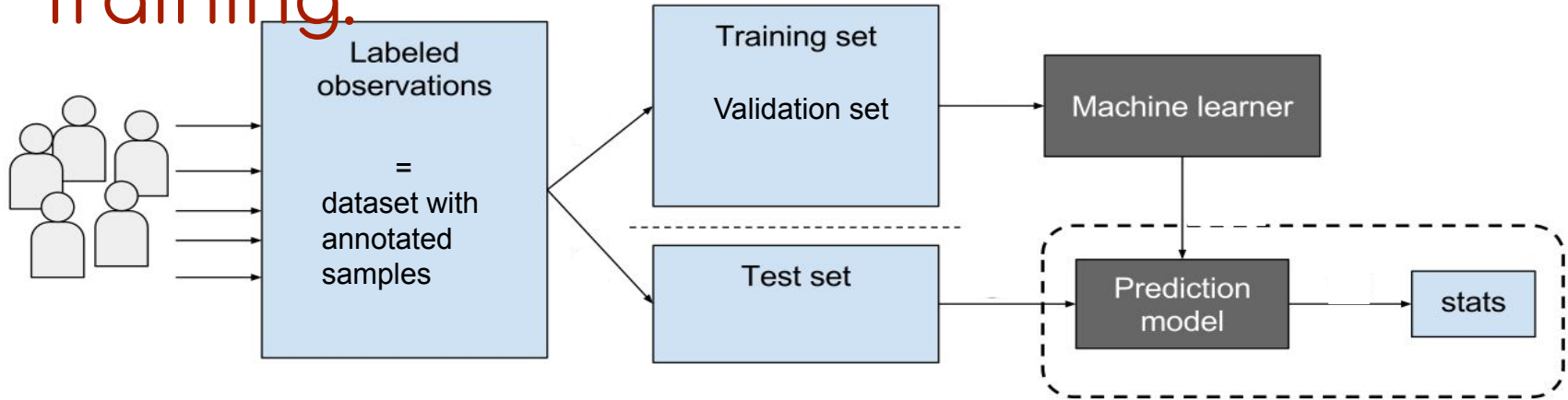
Learning Paradigms



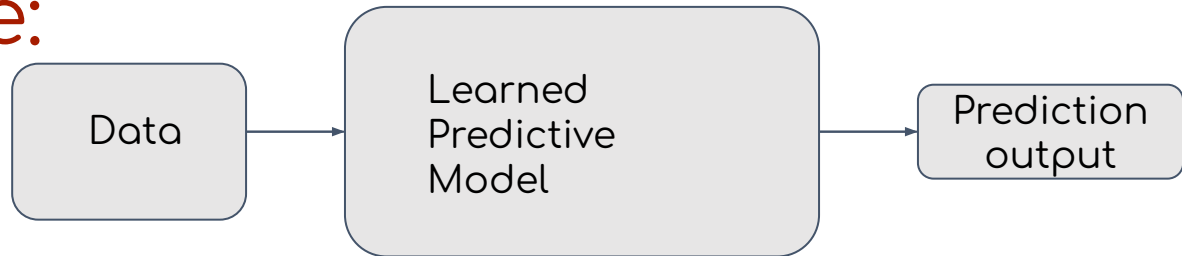
- **Supervised learning**
fully supervised, semi-supervised, weakly supervised and more...
- **Unsupervised learning**
fully unsupervised learning, semi-supervised
- **Reinforcement learning**

Supervised learning - basic concept

Training:



Inference:



Types of supervision

Fully supervision

usage of a **fully labeled** dataset to learn the relationship between input and output data

Weak supervision

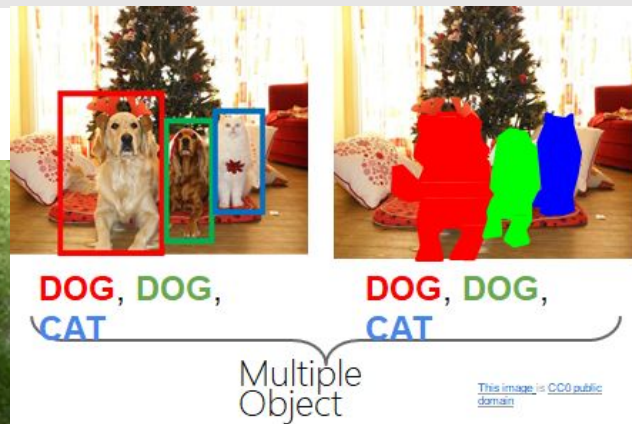
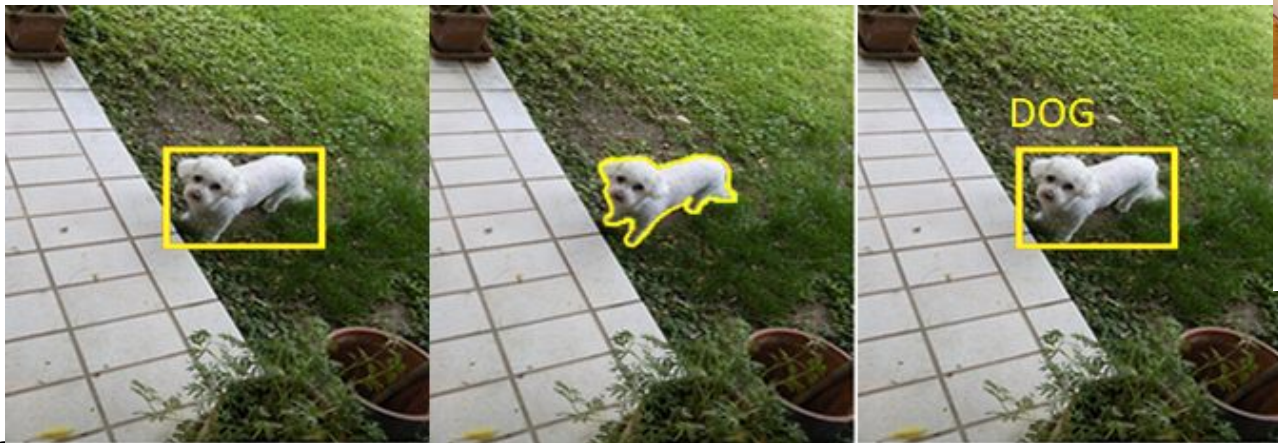
training with labels that may include **mistakes in label annotations**

Semi-supervised

combines partially supervised and partially unsupervised approach by using a **small amount of** labeled data alongside a large amount of unlabeled data to train a model

Object recognition

Type of task



Object detection Object segmentation Object detection and recognition

Example: people detection vs. people recognition, verification



People detection

-> identifying and localizing human subjects within an image, typically represented by bounding boxes around detected individuals



Person recognition

→ Identifying a person's identity from a dataset.

Person verification

→ Confirming whether a claimed identity is correct (Yes/No).

Challenges in object recognition

The recognition method should be invariant to changes in:

scale,
orientation,
illumination,
camera viewpoint,
occlusion,
object pose,
intra-class variations...



Used methods used for object detection & object recognition

Pattern matching

- Rule based pattern recognition

- Statistical pattern recognition

- Fuzzy pattern recognition

Artificial intelligence -> typically classification task

Feature extraction + classification

Neural networks

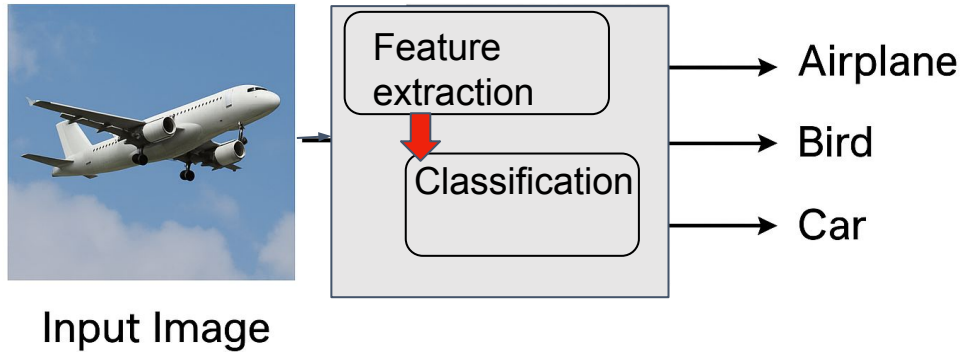
- Methods of Deep learning

 - Convolutional neural networks CNN

 - Vision transformers

Object recognition as a classification task

Traditional Approach of Object Recognition



The task of object recognition within an image is treated as a classification problem.

The process is typically divided into two main stages: feature extraction and classification.

Feature extraction

Feature extraction stage

The goal is to transform raw image data into a set of discriminative features that effectively describe the object's visual characteristics.

In the traditional computer vision pipeline, feature extraction plays a crucial role in transforming raw pixel data into a more meaningful representation that can be used for object recognition or classification.

Features aims to describe the object by description of:

- Texture
- Color
- Shape

Examples:

Color histograms , Gabor filters, HOG (Histogram of Oriented Gradients)



Input Image

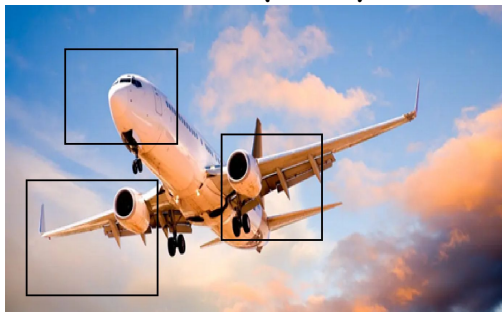
$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Feature vector

Feature extraction

Input:

Image, Region of Interest (ROI), or a tile/patch of the image



Output:

Feature vector or feature matrix



Input Image

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Feature vector

Feature extraction using convolution

Handcrafted (Manually Designed) Filters

One common approach of feature extraction involves using convolution operations with manually designed (handcrafted) filter kernels.

Feature extraction using convolution is based on the principle that important visual characteristics — such as edges, corners, and textures — can be detected using filtration by various filters.

Before deep learning, filter kernels were not learned from data. Instead, computer vision experts **manually designed filters based on known mathematical and perceptual properties to detect specific types of features.**

Some common handcrafted filters include:

- Sobel / Prewitt Detect horizontal or vertical edges
- Laplacian Detect edges and fine details (second-order derivative)
- Gabor Filter Capture oriented textures and frequency patterns
- Tunable sinusoidal kernel
- Gaussian Filter Perform smoothing and reduce noise
- Haar-like Features Detect contrast-based patterns (used in face detection)

Classification

Classification stage

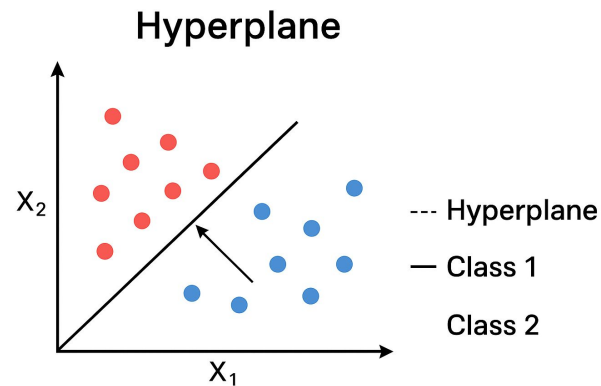


The extracted features are passed into a classifier that decides which class the object belongs to.

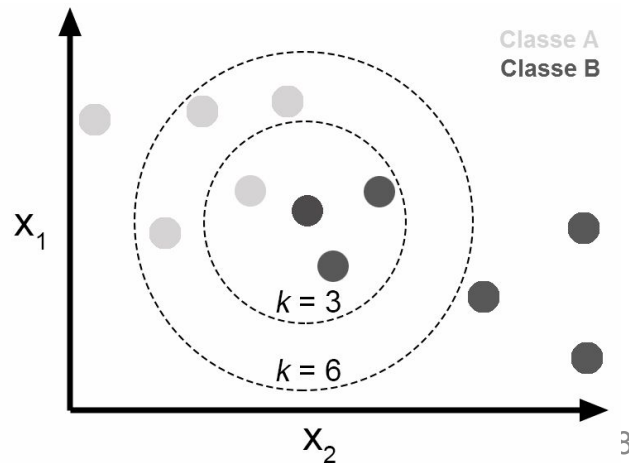
A machine learning algorithm like Support Vector Machine (SVM), k-NN, Random Forest, Bayes Classifier, MLP or other is trained using labeled feature vectors derived from data.

Classifier examples

Simple Linear Classifier

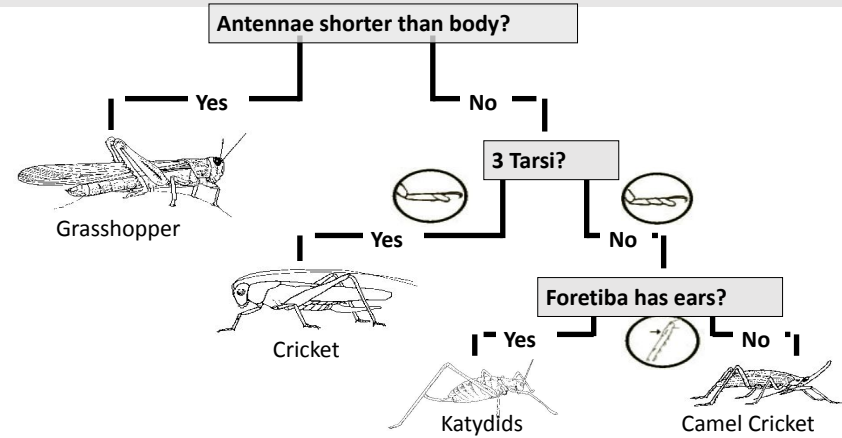


k-nearest neighbours (k-NN)



Classifier examples

Decision Tree Classifier



The Random Forests

Given a training set S .
for $i = 1$ to k :

Build **subset** S_i by sampling with replacement from S , learn tree T_i from S_i

Make predictions according to majority vote of the set of k trees.



k : number of trees T

Classifier examples

AdaBoost Algorithm

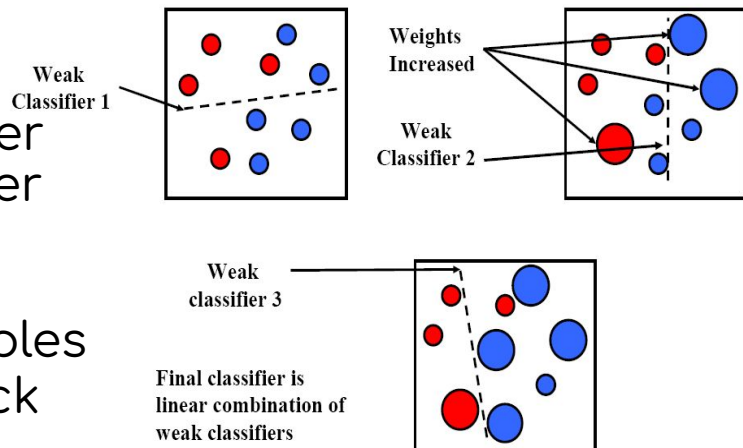
Build a strong classifier by combining number of “weak classifiers”, which need only be better than 50%

Start with uniform weights on training examples
Evaluate weighted error for each feature, pick best.

Incorrectly classified -> more weight

Correctly classified -> less weight

Final classifier is linear combination of the weak ones, weighted according to error they had.



Classifier examples

Support vector machine SVM

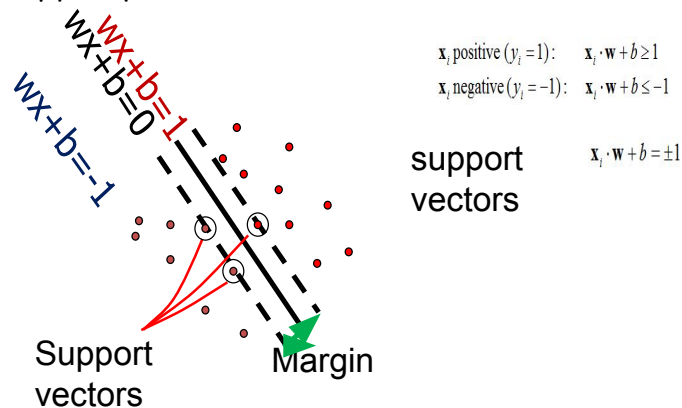
A support vector machine constructs a hyperplane or set of hyperplanes in a high-dimensional space, which can be used for classification, regression or other tasks.

Consider a binary classification problem $(-1, 1)$

Bayes classifier

is a probabilistic classifier based on applying Bayes' theorem (from Bayesian statistics)

hyperplane that maximizes the margin



$$p(h | d) = \frac{P(d | h) P(h)}{P(d)}$$

$P(h)$:

prior belief (probability of hypothesis h before seeing any data)

$P(d | h)$:

likelihood (probability of the data if the hypothesis h is true)

$P(d) = \sum_h P(d | h) P(h)$:

data evidence (marginal probability of the data)

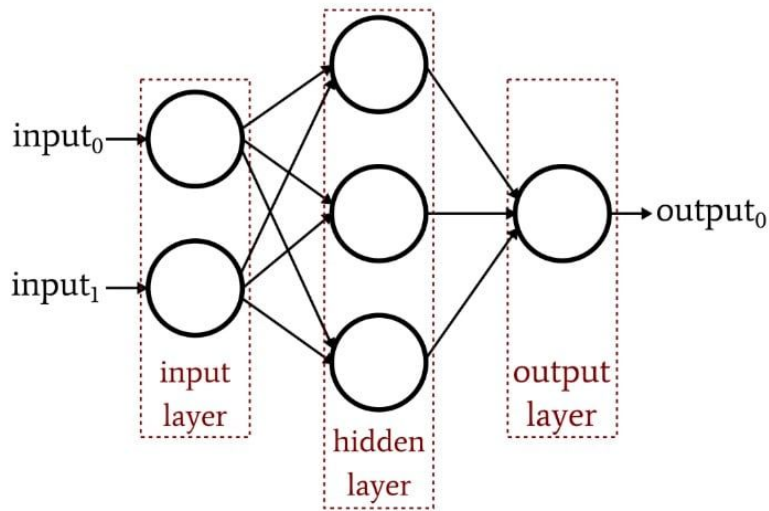
$P(h | d)$:

posterior (probability of hypothesis h after having seen the data d)

Classifier examples

MLP

When applied to classification, the MLP learns to map input feature vectors to specific class labels by discovering complex nonlinear decision boundaries through multiple layers of interconnected neurons.

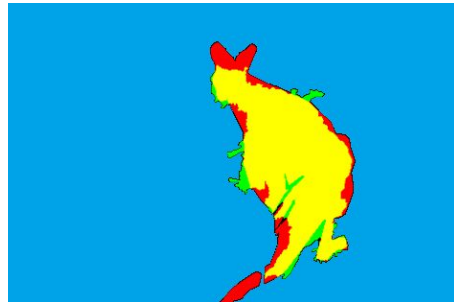


Evaluation of classification

true positive (TP), true negative (TN) false positive (FP), false negative (FN)

groundTruth:

segmentation result:



Task: Object/background segmentation

TP : Number of true predicted pixels from all real positive cases in the data (yellow)

TN : Number of true predicted pixels from all real negative cases in the data (blue)

FP : Number of false predicted pixels from all real positive cases in the data (red)

FN : Number of false predicted pixels from all real negative cases in the data (green)

Precision and recall

precision or positive predictive value (PPV)

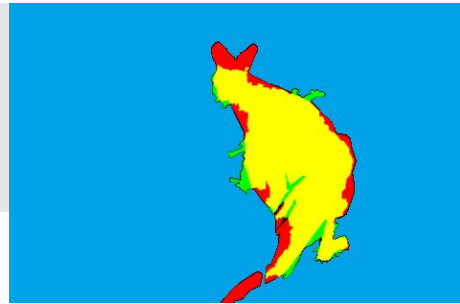
$$PPV = \frac{TP}{TP + FP}$$

Precision is the fraction retrieved that are relevant to the user's information need.

recall, sensitivity, hit rate, or true positive rate (TPR)

$$TPR = \frac{TP}{TP + FN}$$

Recall is the fraction that are relevant to the query that are successfully retrieved.



TP : Number of true prediction from all real positive cases in the data (yellow)

TN : Number of true prediction from all real negative cases in the data (blue)

FP : Number of false prediction from all real positive cases in the data (red)

FN : Number of false prediction from all real negative cases in the data (green)

Accuracy (ACC), F1 score

Accuracy (ACC)

$$ACC = \frac{TP + TN}{P + N}$$

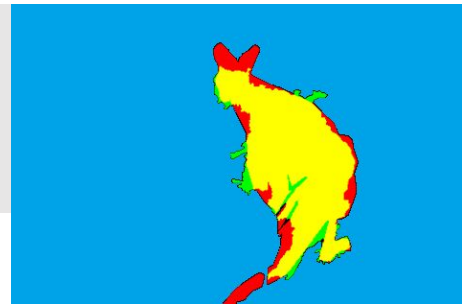
$$P = TP + FN$$

$$N = FP + TN$$

The F1 score is the harmonic mean of the precision and recall.

$$F_1 = 2 * \frac{\text{precision} * \text{recall}}{\text{precision} + \text{recall}}$$

$$F_1 = \frac{2TP}{2TP + FP + FN}$$



TP : Number of true prediction from all real positive cases in the data (**yellow**)

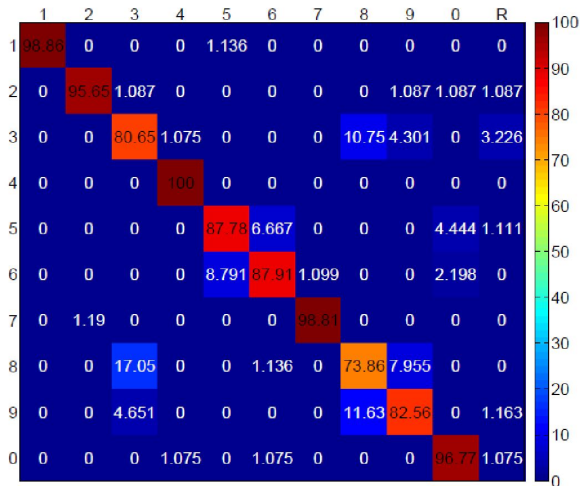
TN : Number of true prediction from all real negative cases in the data (**blue**)

FP : Number of false prediction from all real positive cases in the data (**red**)

FN : Number of false prediction from all real negative cases in the data (**green**)

Confusion matrix - error matrix (Matica zámén)

Each column of the matrix represents the instances in a predicted class while each row represents the instances in an actual class (or vice versa).



Binary classification:

		Observation	
		Yes	No
Prediction	Yes	a true positive hit success	b false positive false alarm Type I error
	No	c false negative miss Type II error	d true negative correct negative correct rejection

Receiver Operator Characteristic - ROC

Receiver Operator Characteristic - ROC

Plot “True Positive Rate TPR” vs. “False Positive Rate FPR”

$$\text{TPR} = \text{TP} / \text{P}$$

$$\text{P} = \text{TP} + \text{FN} \Rightarrow \text{TPR} = \text{TP} / (\text{TP} + \text{FN})$$

Also called hit rate, recall, sensitivity

$$\text{FPR} = \text{FP} / \text{N}$$

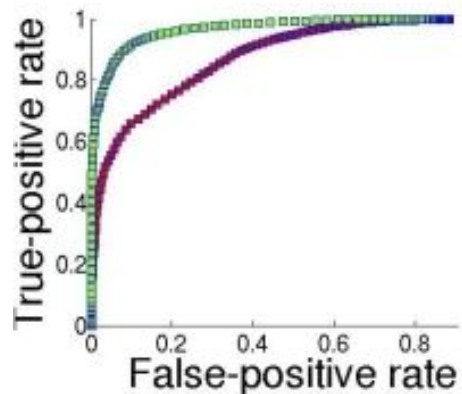
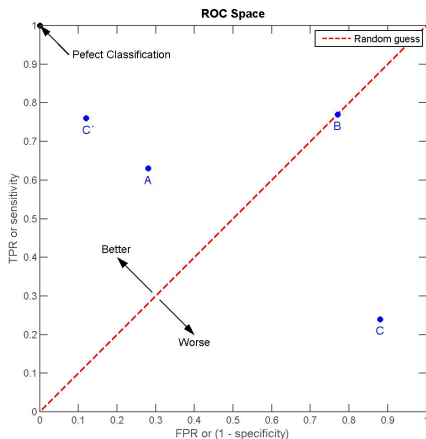
$$\text{N} = \text{FP} + \text{TN} \Rightarrow \text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

Also called false alarm rate, fall-out, =1-specificity

Receiver Operating Characteristic - ROC

Receiver Operating Characteristic curve, or ROC curve, is a graphical plot that illustrates the performance of a binary classifier system as its **discrimination threshold** is varied.

The curve is created by plotting the true positive rate (TPR) against the false positive rate (FPR) at various threshold settings.

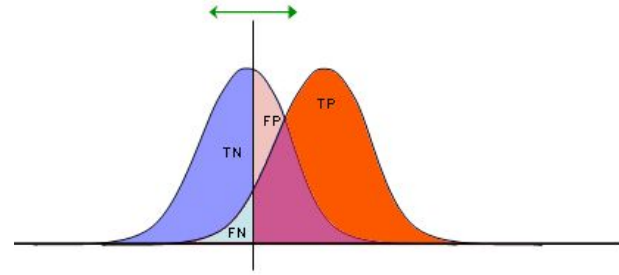


ROC curve example

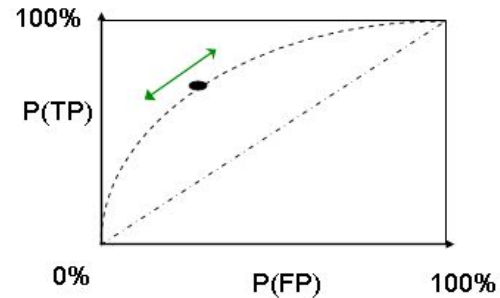
ROC Curve, Area Under Curve (AUC)

AUC stands for "Area under the ROC Curve."

AUC measures the entire two-dimensional area underneath the entire ROC curve (think integral calculus) from (0,0) to (1,1).



TP	FP
FN	TN
1	1



Next parameters for evaluation of classifier

- Time consumption:
 - Time to **construct** the classifier
 - Time to **use** the model (run time)
- Scalability
- Robustness
- Noise
- Missing values

- Interpretability (if possible)

Basic approaches of object detection and object recognition

Basic approaches of object detection and object recognition

Traditional approach based on classification task:

- Global approach : Sliding Windows
- Local : Local descriptors

Era of Deep Learning:

- Convolutional Neural Network (CNN)
- Vision Transformer

Global approach

Global Appearance & Sliding Windows

Sliding window

Size of the window?

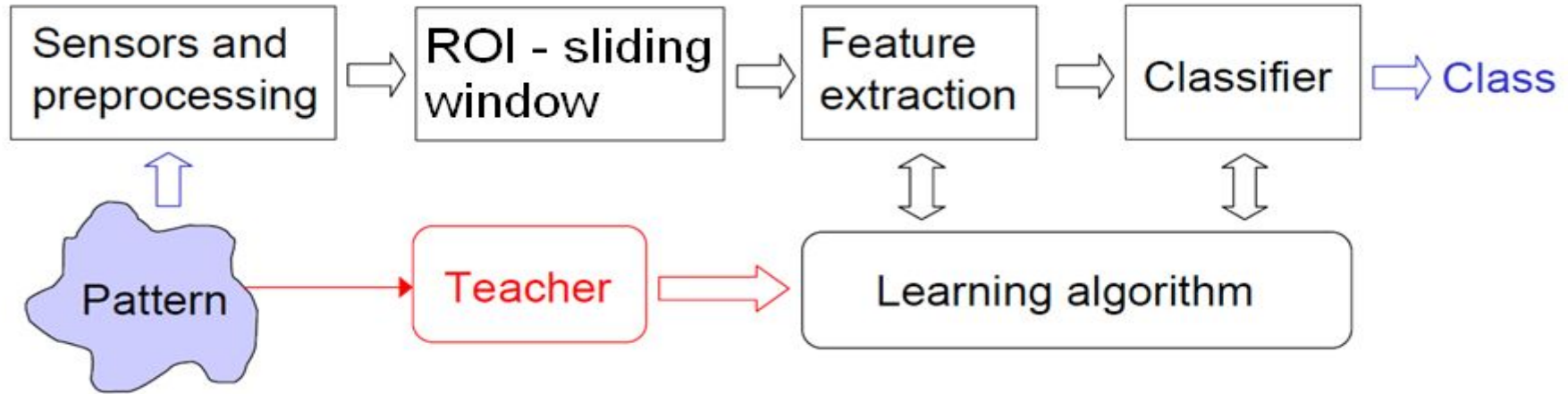
Orientation of the window?



Examples: Face detection,
People detection,



Global Appearance & Sliding Windows



ROI: Region of Interest

Global Appearance & Sliding Windows

Object detection

For object detection, the classification task is a binary decision.

The question that answers classifier:

„Is in the given window the object? (yes or no?)“

! Important is choice of calculated features and also important is choice of a classifier!

Global Appearance & Object recognition

ROI Selection:

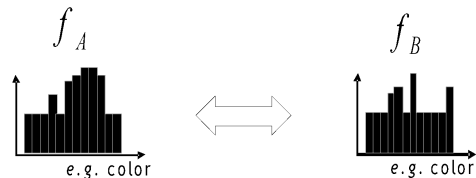
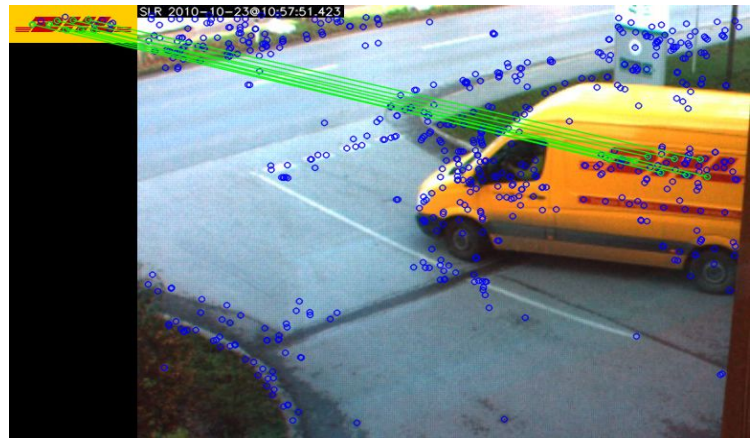
1. The Region of Interest (ROI) is first proposed or identified. It can be manually selected (e.g., bounding box drawn by a user), or automatically proposed using algorithms such as:
 - a. Sliding window
 - b. Selective Search : Edge boxes[1], methods using segmentation, Bounding box proposals and more..
2. Feature extraction and classification of the object (class)

[1]ZITNICK, C. Lawrence; DOLLÁR, Piotr. Edge boxes: Locating object proposals from edges. In: *European conference on computer vision*. Cham: Springer International Publishing, 2014. S. 391-405.

Local : Local descriptors

Object detection using local descriptors

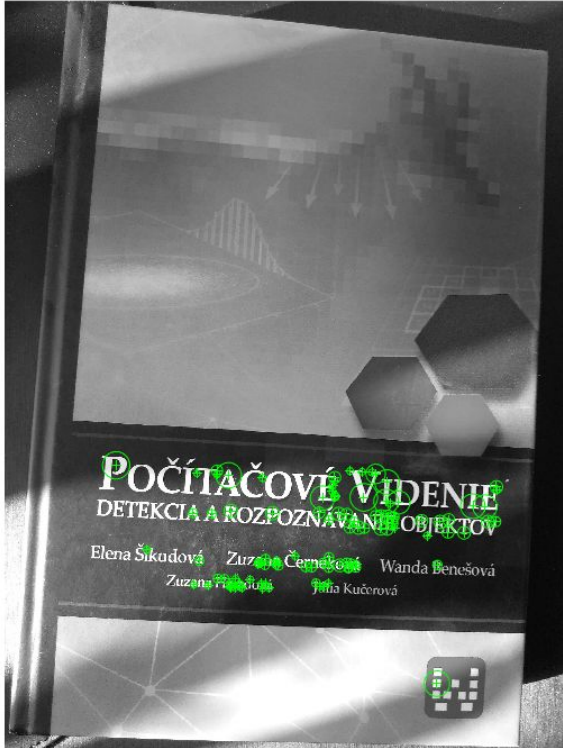
1. Find distinctive key-points using key-point detectors and define a region around each keypoint
2. Compute a local description for the normalized region using key-point descriptors
3. Match the local descriptors
4. Filter the of inlier/outlier
5. Calculate homography



$$d(f_A, f_B) < T$$

Key points detection and description

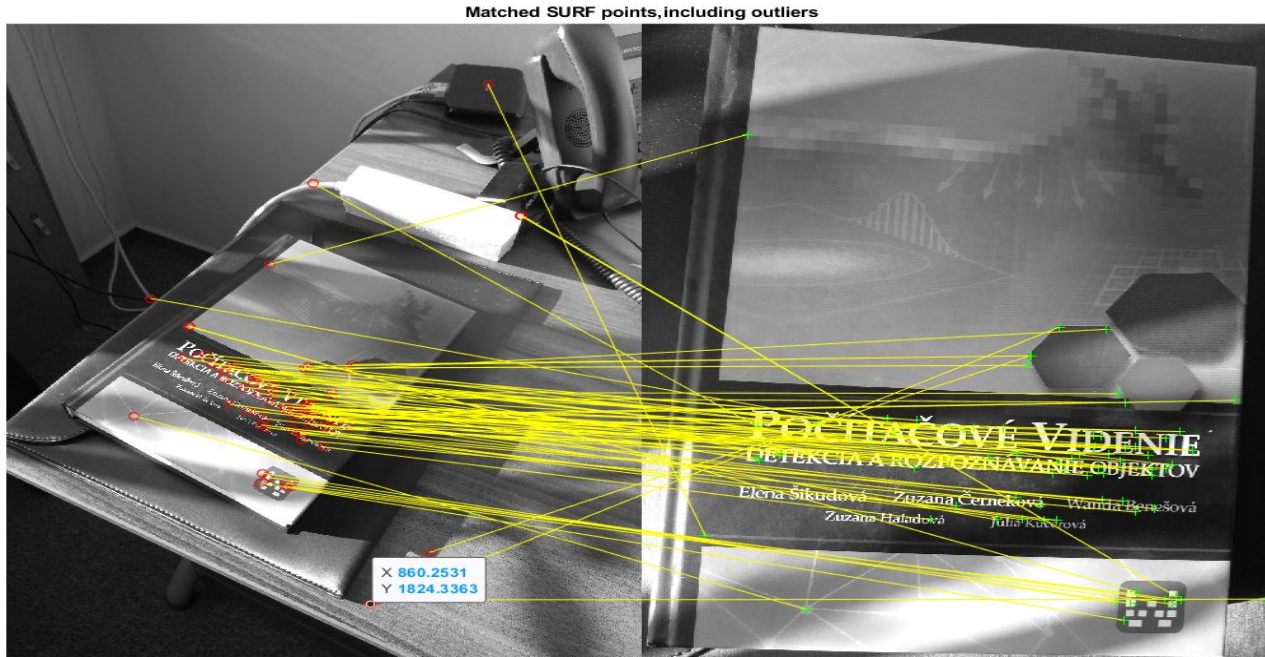
100 Strongest Feature Points from Box Image



300 Strongest Feature Points from Scene Image



Match local descriptors



Filtering of inlier/outlier

Matched inlier points



6. Calculate homography

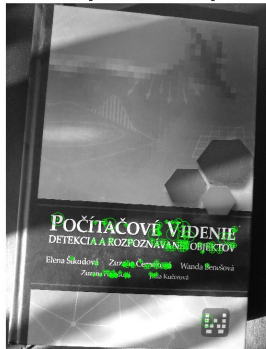
Result after homography transform

Visualisation:

Template projection – blue channel

Original Image – red channel

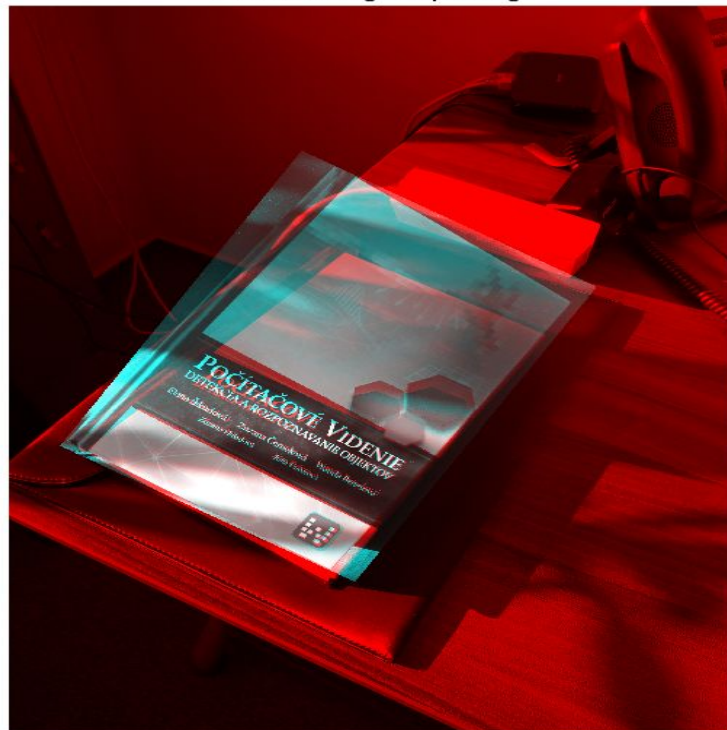
100 Strongest Feature Points from Box Image



300 Strongest Feature Points from Scene Image



Recovered image + Input Image



Local descriptors

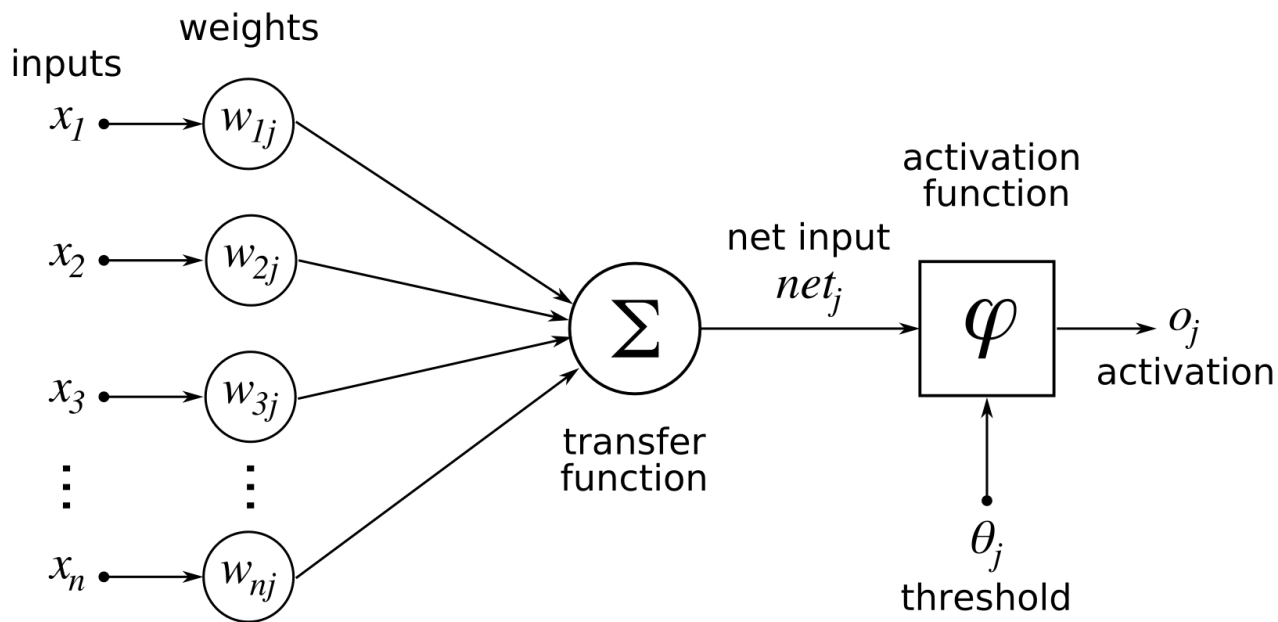
More robust than global approach, better from the point of view for:

- occlusions of objects
- changes of camera view position
- rotation, scale invariance
- intra category variations

Basics of Neural Networks (NN)

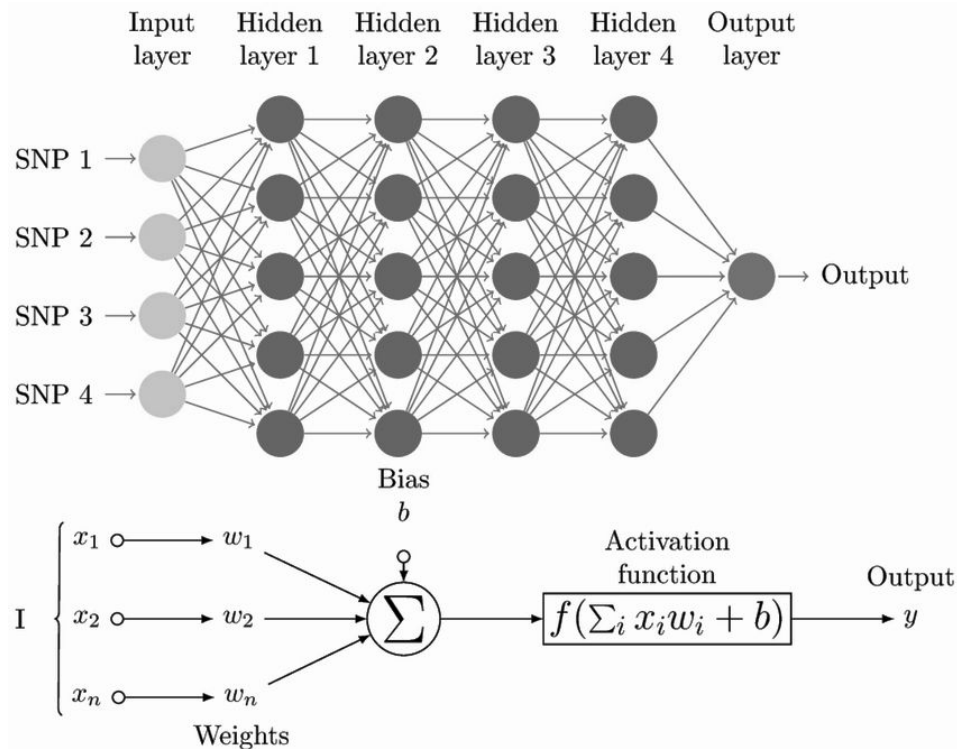
Model of an artificial neuron

Neuron



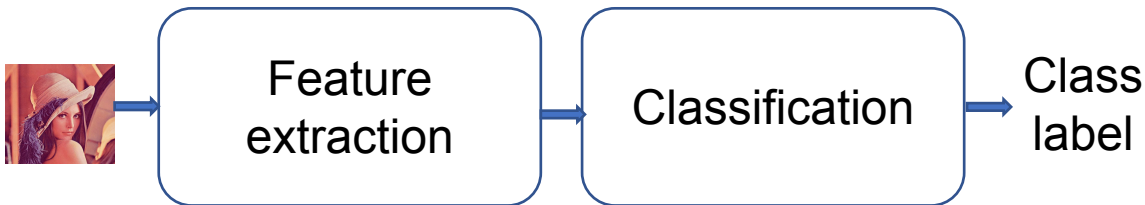
MLP Multilayer Perceptron

Multi-Layer Perceptron (MLP)
diagram with four hidden
layers



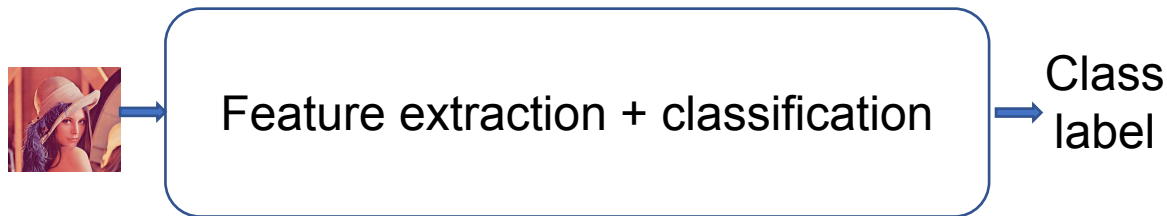
Deep Learning methods

Classification task and Deep Learning



- Traditional approach

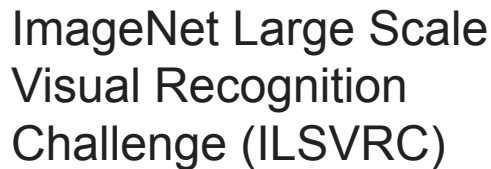
Suboptimal, requires expert knowledge, hand crafted features



- CNN - Deep learning

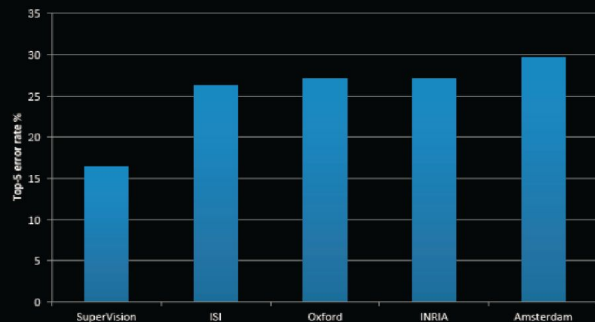
both the classifiers and the features are learned automatically

Imagenet database: 14 M labeled images, 20K categories



Deep convolutional neural networks

- Krizhevsky et al. -- 16.4% error (top-5)
- Next best (non-convnet) – 26.2% error



Krizhevsky, Sutskever, Hinton: ImageNet classification with deep convolutional neural networks. NIPS, 2012.

Recognizes 1000 categories from ImageNet

Outperforms state-of-the-art by significant margin (ILSVRC 2012)

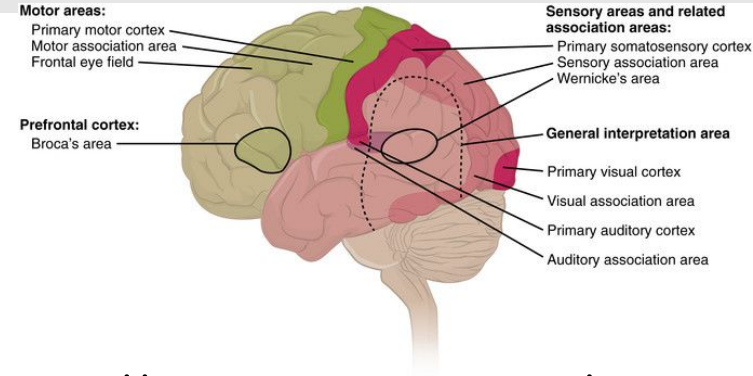
Convolutional Neural Network

Convolutional Neural Network

Inspired by biology

Visual cortex - topographical representation of the field of vision

- Reception field of neighbouring cells corresponds to adjacent portions of the field of vision
- Sequence of cells can perceive complex shapes
- Uses convolution as building blocks



2D cross-correlation & convolution

Correlation $\text{Corr}(f(x, y) \ g(x, y)) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n)g(x + m, y + n)$

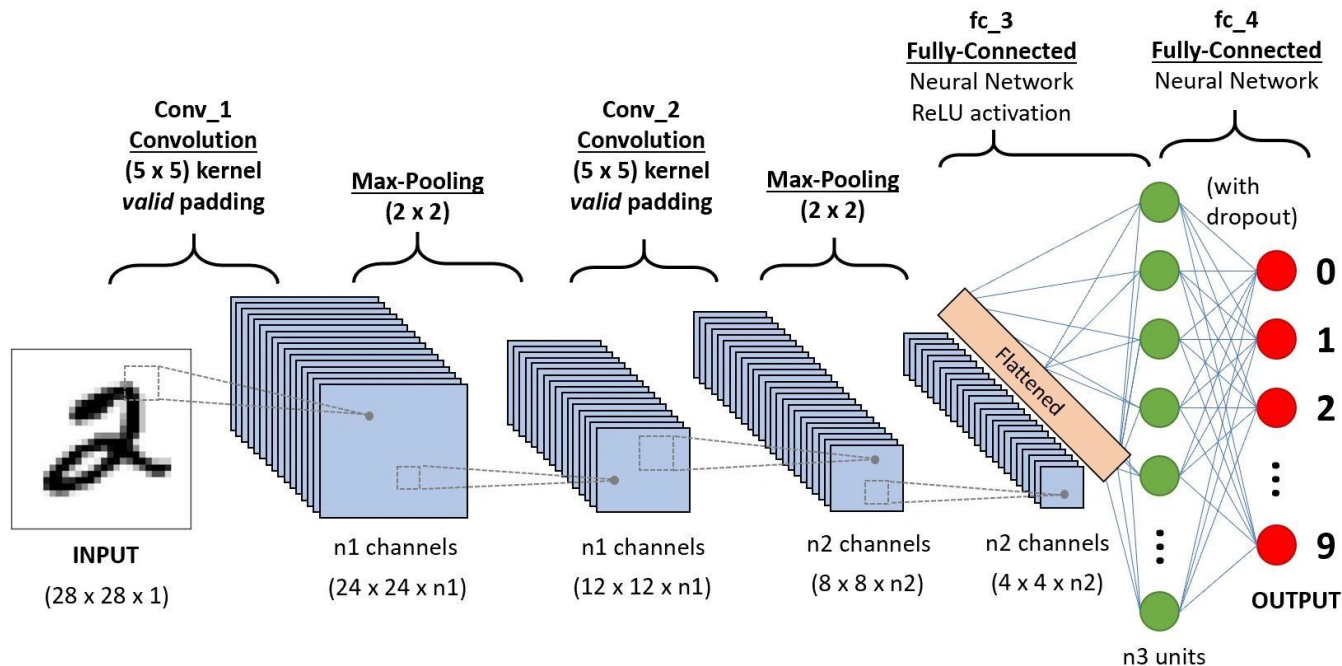
Convolution $f(x, y) * g(x, y) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n)g(x - m, y - n)$

The kernel is learned -> we do not flip the kernel ->

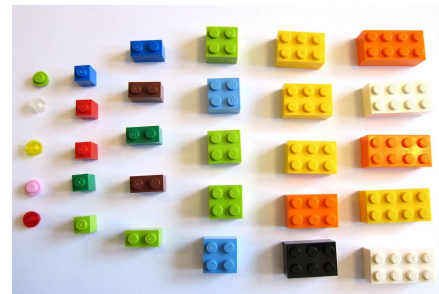
Cross-correlation is easier to implement for learned kernels.

We call it “Convolution” 😊

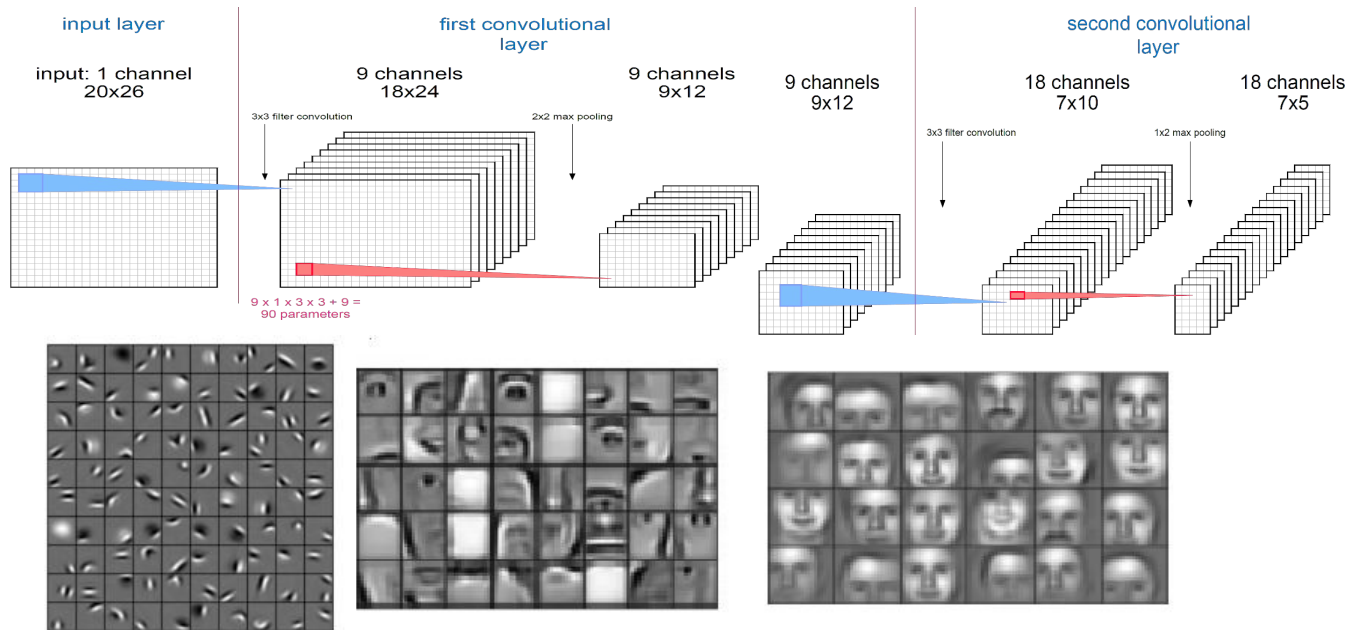
CNN for MNIST dataset digit recognition



Demo



Convolutional layers in CNN - example



Learned hierarchical features from a deep learning algorithm.

Architecture of CNN

Layers in CNN

Layers in CNN

Input image

Convolution layer (Convol.)

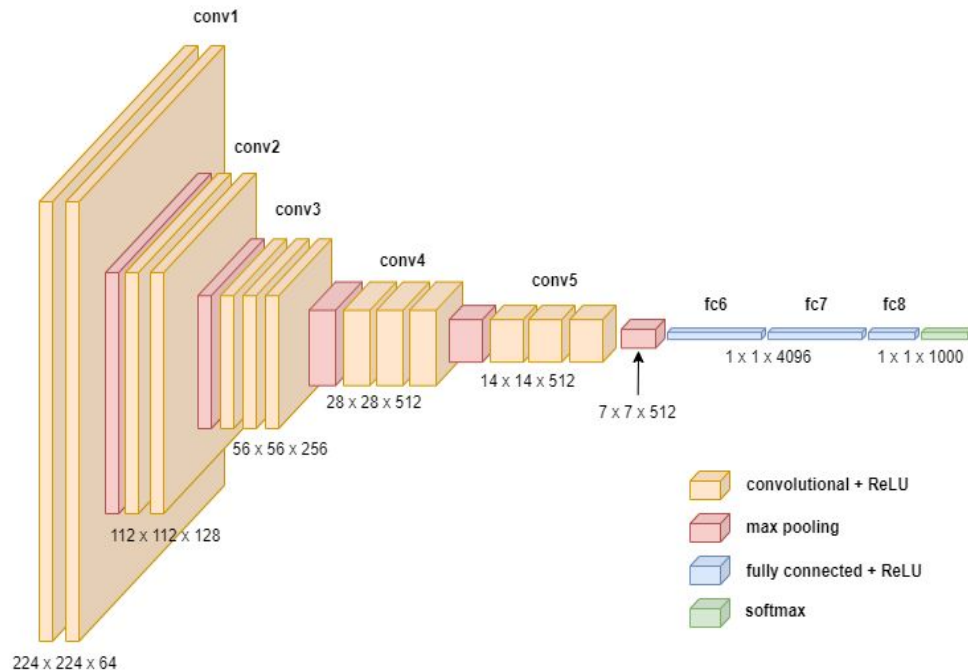
Activation functions (ReLU)

Pooling layer (max pooling)

Fully Connected (Fully con)

Softmax

Output



Input layer

Accepts raw image data and feeds it into the network.

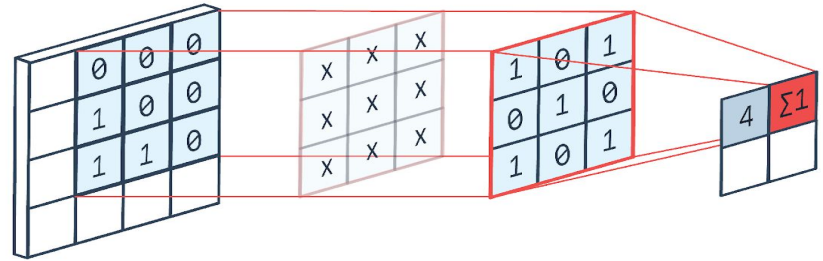
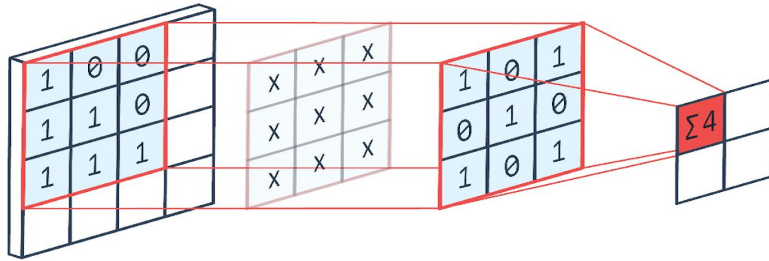
Input image could be a multichannel image -> for a color image, each pixel is represented by 3 channels (R, G, B).

Example:

An image of size 128×128 pixels has an input **tensor** shape of (128, 128, 3).

Convolution layer-core building block of CNNs.

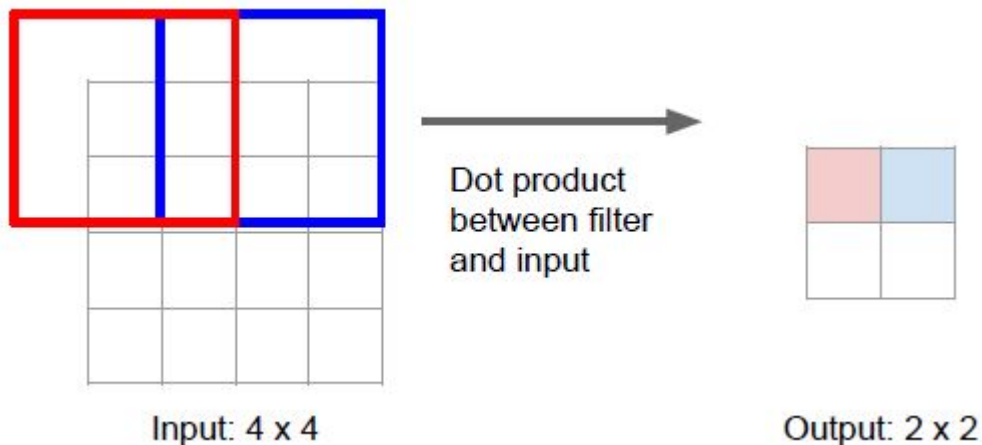
Extracts local spatial features such as edges, textures, corners, and patterns using learnable filters (kernels) - each filter is matrix of trainable weights



- Each filter (a small matrix, e.g., 3x3 or 5x5) slides across the image and performs a convolution operation, computing the dot product between the filter and the image region.
- each convolution layer is composed by a configurable number of filters
- convolution operation is performed between the image and each filter, producing as output a new tensor

Strided convolution

example: 3 x 3 convolution, stride 2 pad 1

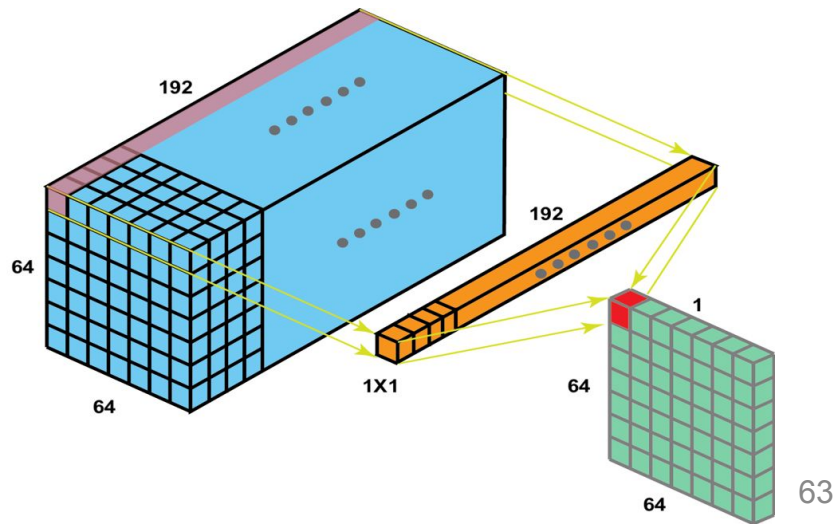


Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

1x1 convolution

A 1x1 convolution is a convolutional neural network operation that applies a 1x 1 kernel across the input, essentially acting as a mini-neural network at each pixel's location **to combine features across channels**.



Padding in CNN

0	0	0	0	0	0	0	0
0	3	3	4	4	7	0	0
0	9	7	6	5	8	2	0
0	6	5	5	6	9	2	0
0	7	1	3	2	7	8	0
0	0	3	7	1	8	3	0
0	4	0	4	3	2	2	0
0	0	0	0	0	0	0	0

$6 \times 6 \rightarrow 8 \times 8$

*

1	0	-1
1	0	-1
1	0	-1

3×3

=

-10	-13	1			
-9	3	0			

6×6

Example of zero padding $p=1$.

Applying padding of 1 before convolving with 3×3 filter

Activation Layer -> Non-Linearity Layer

Every value in the tensor is fed through an activation function to introduce a nonlinearity.

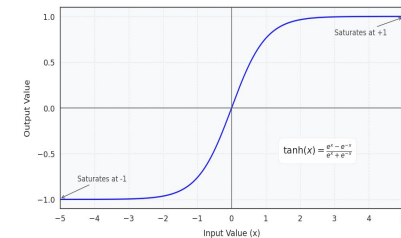
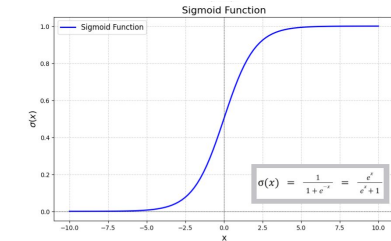
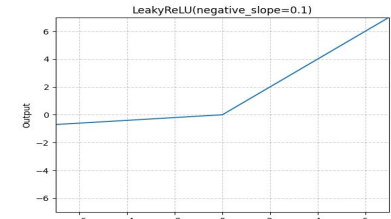
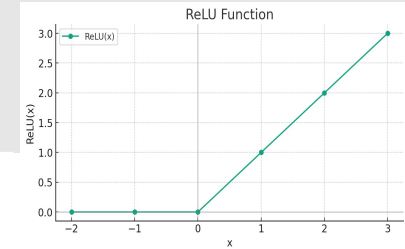
If you remove the activation functions, the entire network becomes a stack of linear operations.

Real-world image data contain non-linear relationships and linear operations alone cannot model these complexities.

Activation functions allow CNNs to build hierarchical, non-linear representations.

Common type of activation functions

Function	Formula	Key Property
ReLU (Rectified Linear Unit)	$f(x) = \max(0, x)$	Fast, prevents vanishing gradient
Leaky ReLU	$f(x) = x$ if $x > 0$, else $0.01x$	Avoids dead neurons
Sigmoid	$f(x) = \frac{1}{1 + e^{-x}}$	Maps to $[0, 1]$, used in binary output
Tanh	$f(x) = \tanh(x)$	Zero-centered output

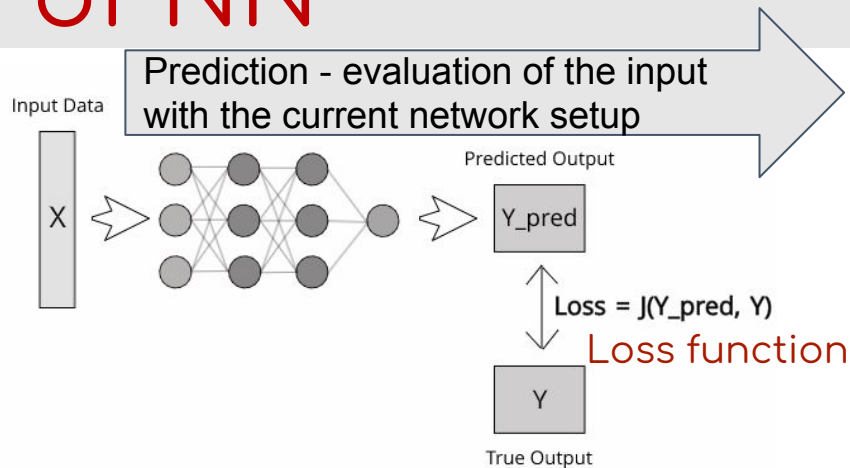


Parameters of Convolution filters

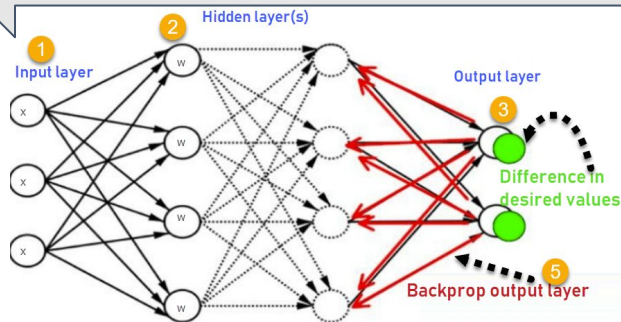
- **Number of filters:** The number of convolutional filters in the layer
- **Width of filter:** The width of the filter matrix of a single filter.
- **Height of filter:** The width of the weight matrix of a single filter.
- **Horizontal stride:** The number of pixels to move while performing the convolution along the horizontal axis.
- **Vertical stride:** The number of pixels to move while performing the convolution along the vertical axis.
- **Padding:** Same results in padding the input such that the output has the same length as the original input.
- **Activation function:** The function that will applied to each element of the output. (ReLU, sigmoidal...)

Learning of CNN

Fully supervised learning - training of NN



Training - error back propagation



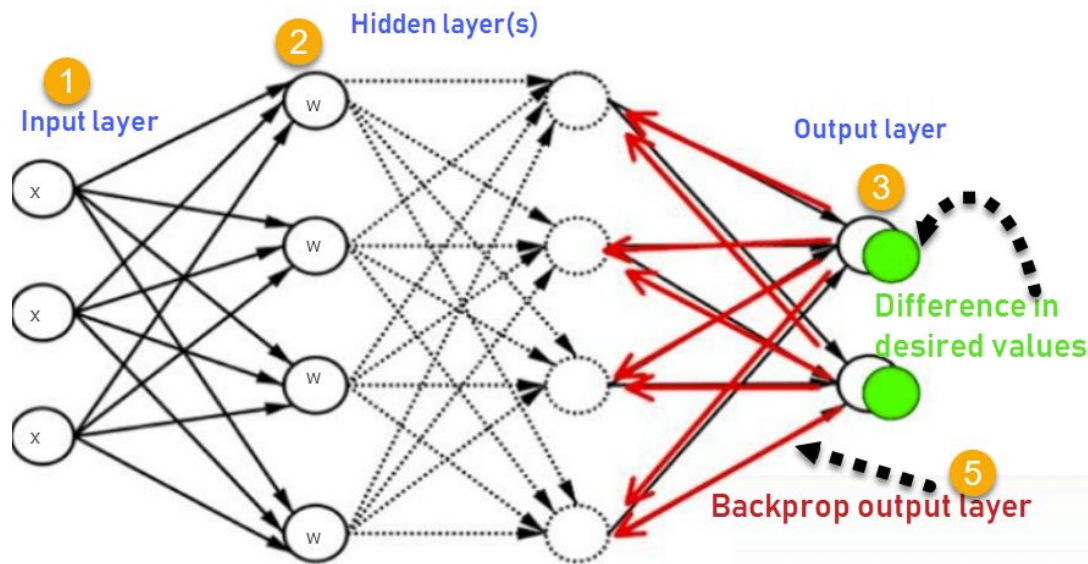
Learning objective:

- The network learns by comparing the input data and the corresponding expected output data.
- Loss function is used for the evaluation of the prediction error.
- Successful learning using error back propagation gradually-iteratively reduces the prediction error

Backpropagation

Computing gradients of expressions through recursive application of chain rule

(Řetízkové pravidlo, řetězové pravidlo neboli pravidlo o derivaci složené funkce)



Loss function vs. cost function

The **loss function** is used to evaluate the error between the predicted value and the true value **for a single record**.

The **cost function** measures the **model's error across a group of records** (i.e., the entire dataset)

Loss functions

Loss Function	Applicability to:	Classification	Regression
Mean Square Error (MSE) / L2 Loss	✗	✓	$MSE = (1/n) * \sum (y_i - \bar{y})^2$
Mean Absolute Error (MAE) / L1 Loss	✗	✓	$MAE = (1/n) * \sum y_i - \bar{y} $
Huber Loss / Smooth Mean Absolute Error	✗	✓	$L(\delta, y, f(x)) = (1/2) * (f(x) - y)^2 \quad \text{if } f(x) - y \leq \delta$ $= \delta * f(x) - y - (1/2) * \delta^2 \quad \text{if } f(x) - y > \delta$
Binary Cross-Entropy Loss / Log Loss	✓	✗	$L(y, f(x)) = -[y * \log(f(x)) + (1 - y) * \log(1 - f(x))]$
Categorical Cross-Entropy Loss	✓	✗	
Hinge Loss	✓	✗	$L(y, f(x)) = \max(0, 1 - y * f(x))$
Log Loss	✓	✗	

Importance of the choice of Loss function

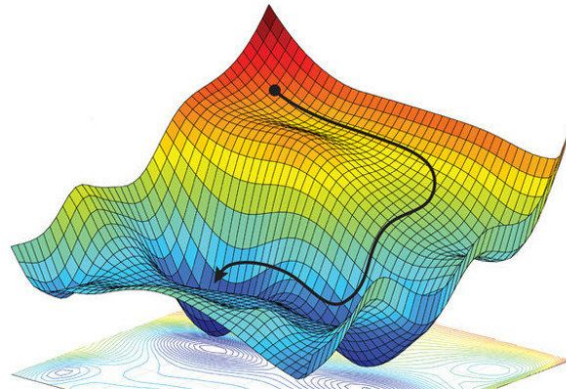
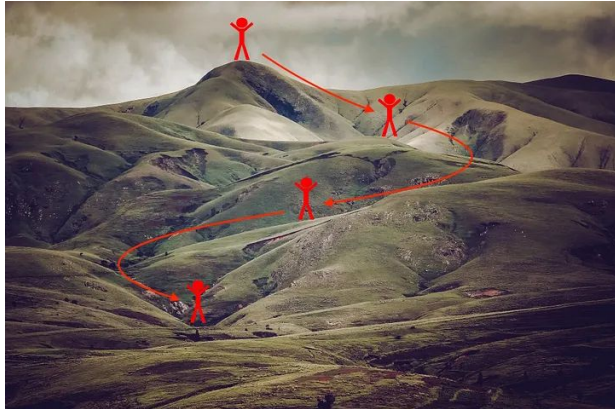
- Guide Model Training:
 - During training, algorithms such as Gradient Descent use the loss function to adjust the model's parameters and try to reduce the error and improve the model's predictions.
- Measure Performance:
 - By finding the difference between predicted and actual values and it can be used for evaluating the model's performance.
- Affect learning behavior:
 - Different loss functions can make the model learn in different ways depending on what kind of mistakes they make.

Optimizers

Update of weights & Gradient descent

Training: we wish that the loss function decrease ->
update the weight vector in negative gradient direction

Procedure of repeatedly evaluating the gradient and then performing a parameter update is called **Gradient Descent**.



Optimize with Gradient Descent

Gradient Descent is an iterative optimization algorithm used to find the minimum of a differentiable function $f(\theta)$ -typically a loss function in machine learning.

The idea:

Move the parameters θ in the opposite direction of the gradient of the function, because the gradient points toward the direction of maximum increase.

Optimize with Gradient Descent

Let $f(\theta)$ be the loss function we want to minimize.

Then, the update rule is:

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta_t)$$

where: θ_t is current parameter vector at iteration t

η is learning rate (a positive scalar step size)

$\nabla_{\theta} f(\theta_t)$ is gradient of f with respect to θ_t

θ_{t+1} is updated parameter vector

Intuitive Meaning: The gradient $\nabla f(\theta_t)$ gives the direction of steepest increase of the function.

Moving in the negative gradient direction reduces the function value - thus, the name descent.

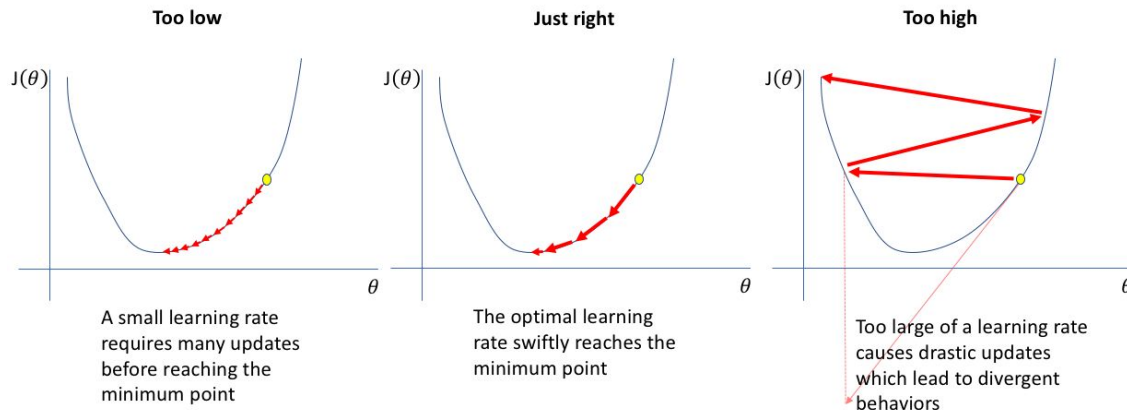
Step size - learning rate η

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} f(\theta_t)$$

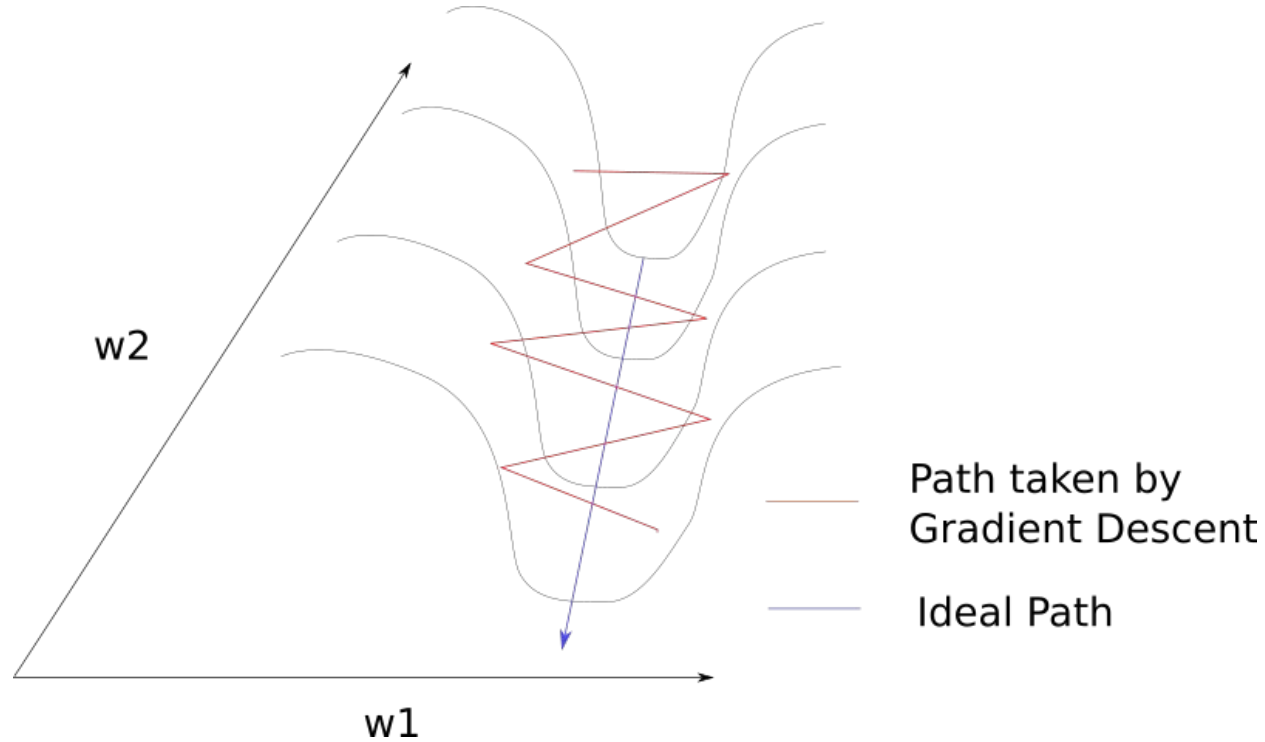
important hyperparameter settings in training a neural network

small step size $\rightarrow$ very small progress

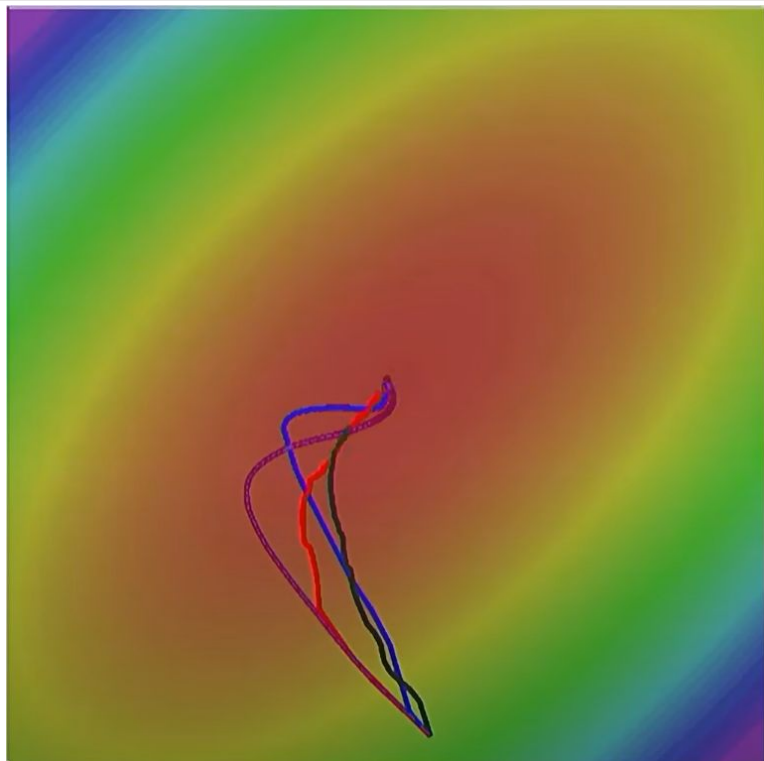
large steps $\rightarrow$ can lead to better progress but are more risky. Note that eventually, for a large step size we will overshoot and make the loss worse



Problem With Gradient Descent



Examples of common optimisers



— SGD

— SGD+Momentum

— RMSProp

— Adam

Stochastic Gradient
Descent (SGD)

Stochastic Gradient
Descent (SGD) +
Momentum

Root Mean Square
(RMS) Propagation

Adam optimiser

Training data batch

Common approach is to compute the gradient over batches of the training data.

Batch size setting - hyperparameter tuning

Batch Size

1. Batch descent

- batch size = M

2. Mini-batch descent

- batch size > 1

3. Stochastic descent

- batch size = 1

Batch Size	
Batch descent	M
Mini-batch descent	BS = 16,32,64,128...
Stochastic descent	1

Samples in mini batch for training must be chosen randomly

Epoch

Neural networks are trained in a series of epochs.

Each epoch consists of one forward pass and one backpropagation pass over all of the provided training samples.

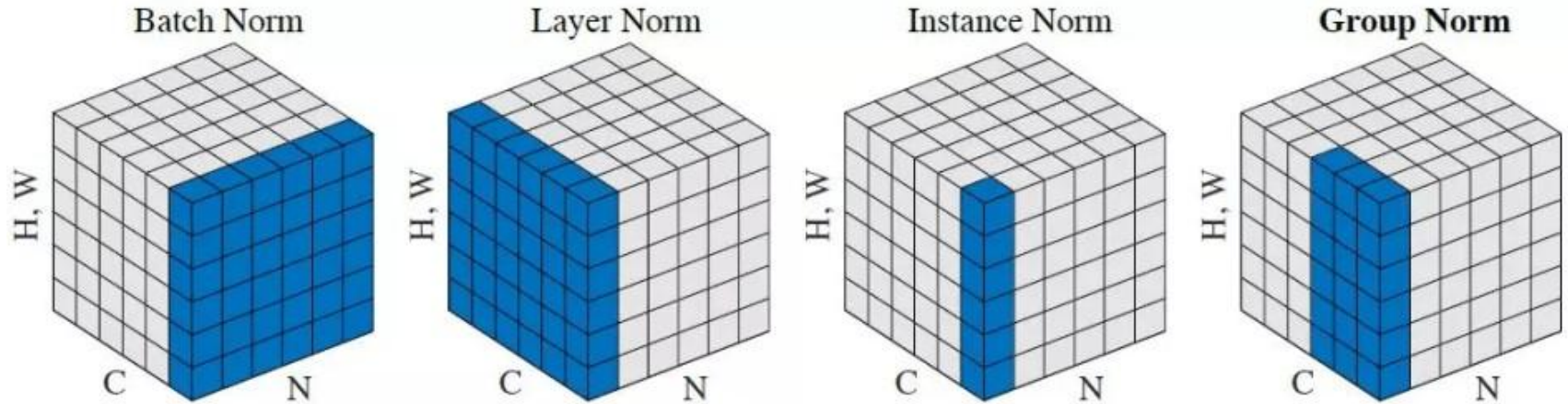
Normalisation

Normalization in a neural network is a technique to scale input data to a standard range, which stabilizes and accelerates the training process.

Common methods include batch normalization, which normalizes activations across a **mini-batch**, and simple **input normalization**, which scales features to a common range like $[0,1]$ or adjusts them to have a mean of 0 and a standard deviation of 1. -> **Z-score normalisation**

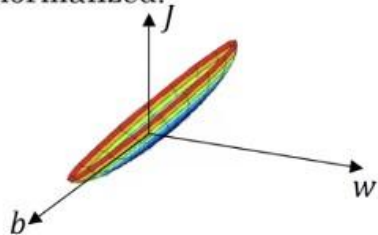
This prevents issues like vanishing or exploding gradients and ensures features with larger scales don't dominate the learning process.

Types of normalization

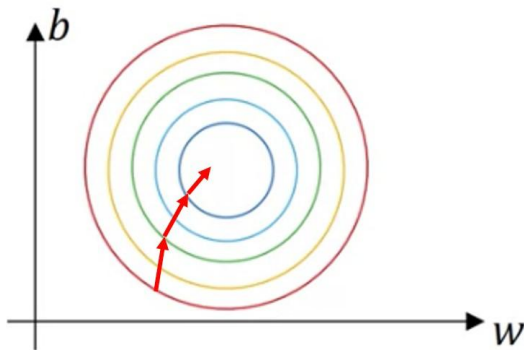
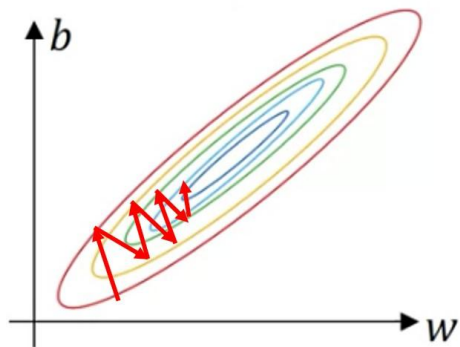
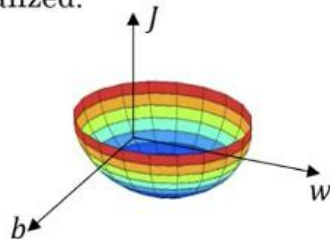


Normalization

Unnormalized:



Normalized:



Statistical Moments

Mean value

$$\mu = E(X) = \frac{1}{m} \sum_{i=1}^m x(i)$$

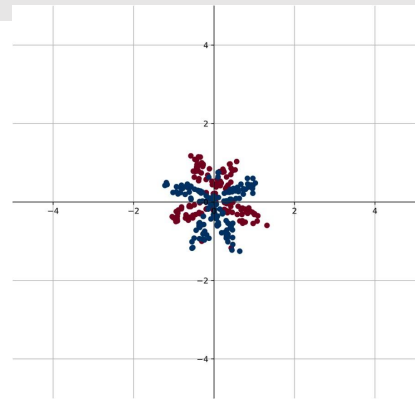
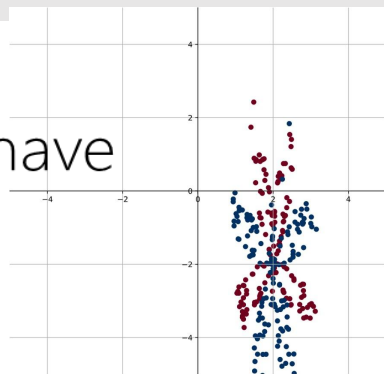
Variance

$$\sigma^2 = \text{Var}(X) = E[(X - \mu)^2]$$

Z-score normalisation

Z-score normalization:

Center and scale a feature to have
mean = 0 and
standard deviation = 1



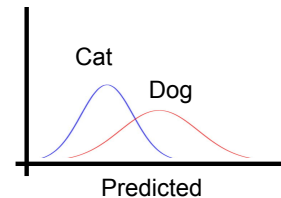
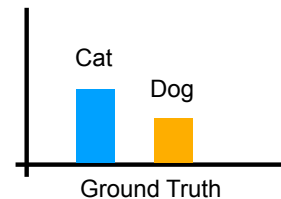
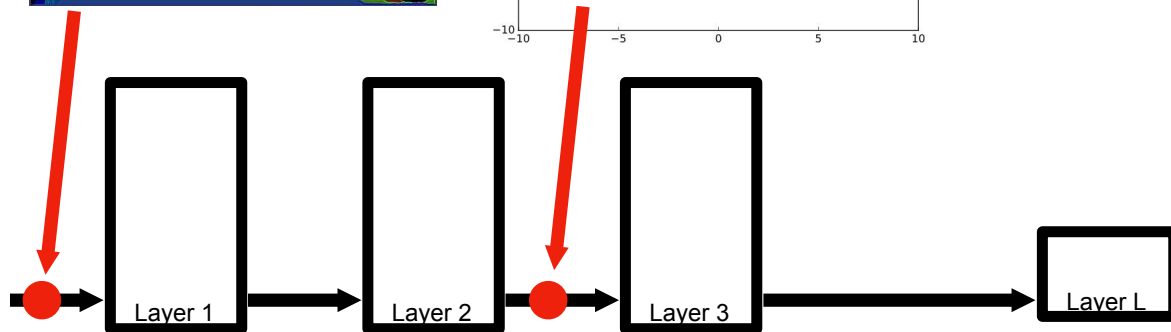
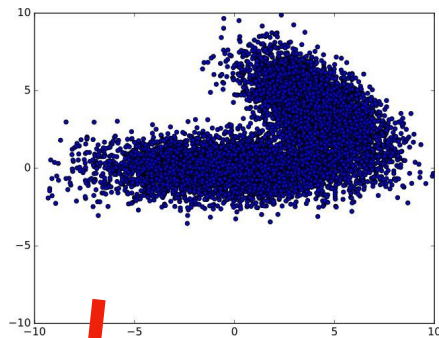
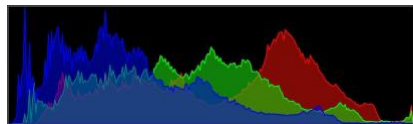
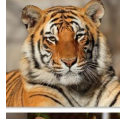
For a random variable X with mean μ and standard deviation σ , the z -score of a value x is :

$$z = \frac{(x - \mu)}{\sigma}$$

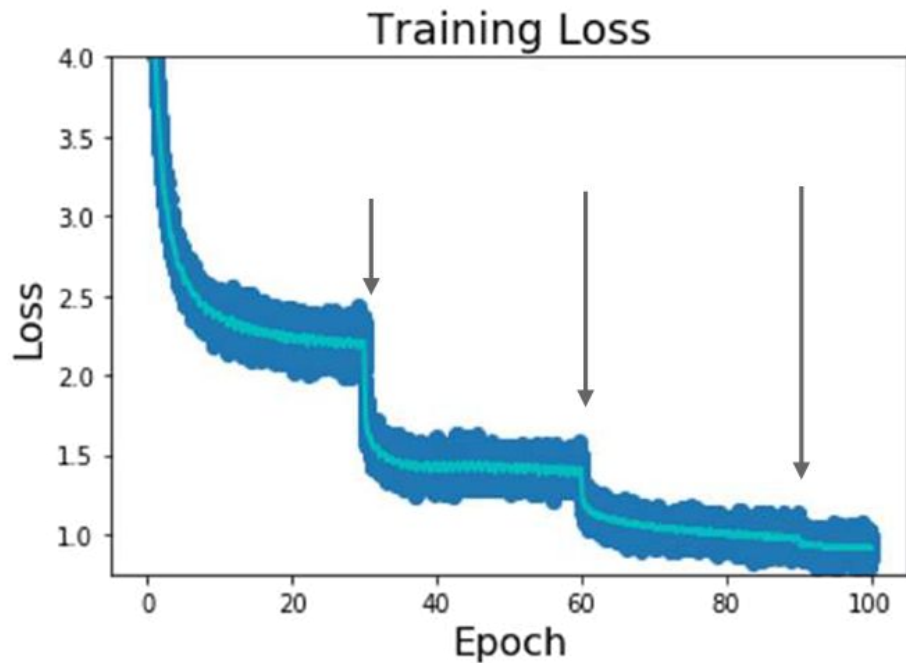
z -scores measure the distance of a data point from the mean in terms of the standard deviation.

The standardized data set has mean 0 and standard deviation 1, and retains the shape properties of the original data set (same skewness and kurtosis).⁸⁸

Normalization



Learning rate (LR) scheduling



Step: Reduce learning rate at a few fixed points.

E.g: multiply LR by 0.1 after epochs 30, 60, and 90.

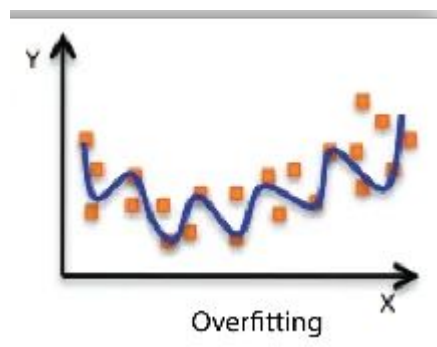
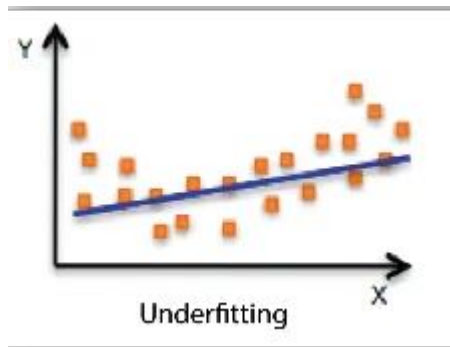
Cosine: $\alpha_t = \frac{1}{2}\alpha_0 (1 + \cos(t\pi/T))$

Linear: $\alpha_t = \alpha_0(1 - t/T)$

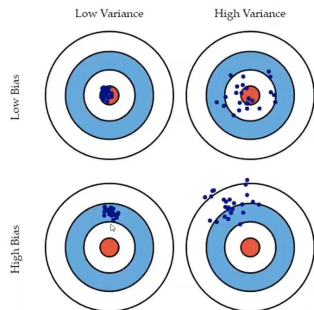
Inverse sqrt: $\alpha_t = \alpha_0/\sqrt{t}$

α_0 : Initial learning rate
 α_t : Learning rate at epoch t
 T : Total number of epochs

Problem of generalization

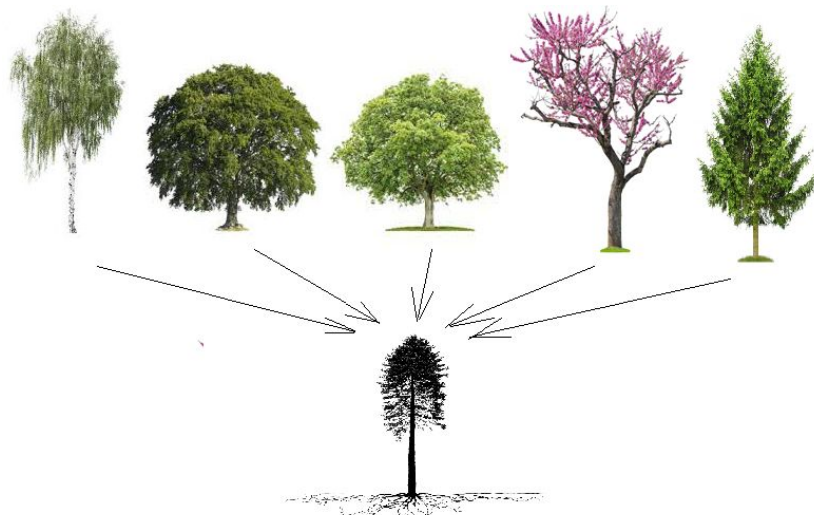
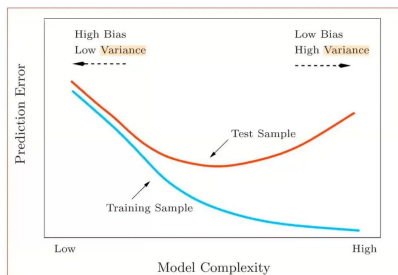


Bias and Variance



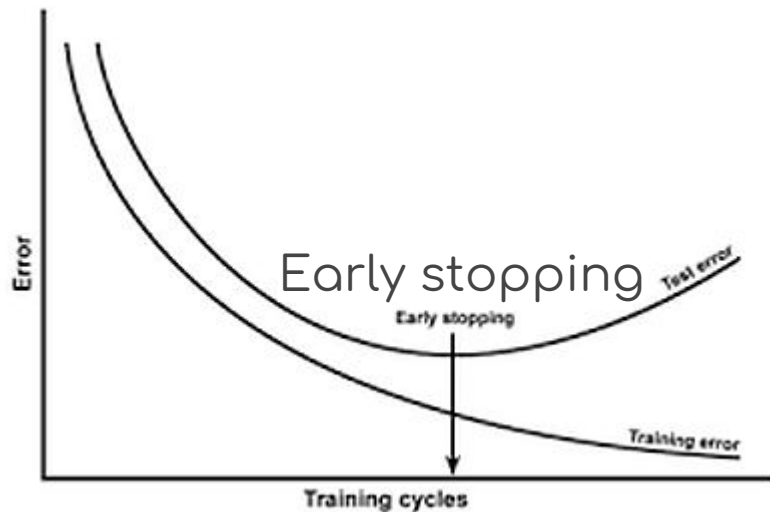
$$\text{Bias} = \mathbb{E}[\hat{f}(x)] - f(x)$$

$$\text{Variance} = \mathbb{E}[(\hat{f}(x) - \mathbb{E}[\hat{f}(x)])^2]$$



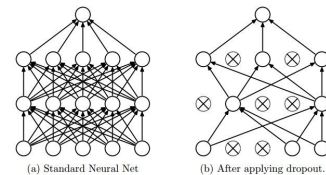
Diagnosing Overfitting

Loss function during the training process



Preventing Overfitting

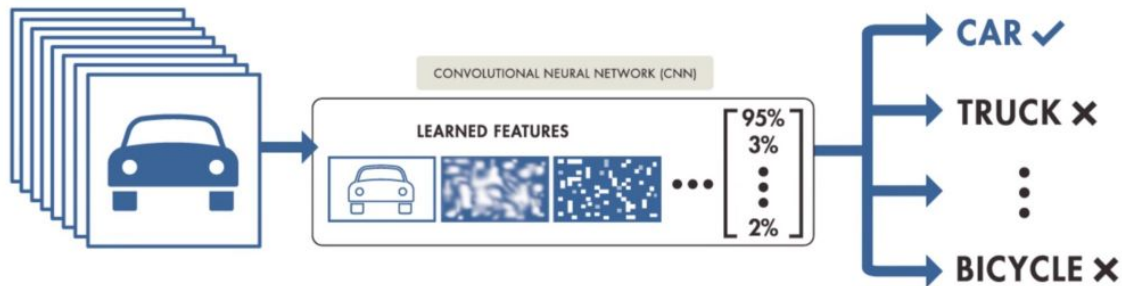
- Training with more data -> Data augmentation
- Ensembling
- Simplify the architecture: Limit the number of hidden layers and the number of units per layer.
- Early stopping
- Weight-decay: Penalize large weights using penalties or constraints on their squared values (L2 penalty) or absolute values (L1 penalty).
- Add noise to the weights or the activities.
- Dropout



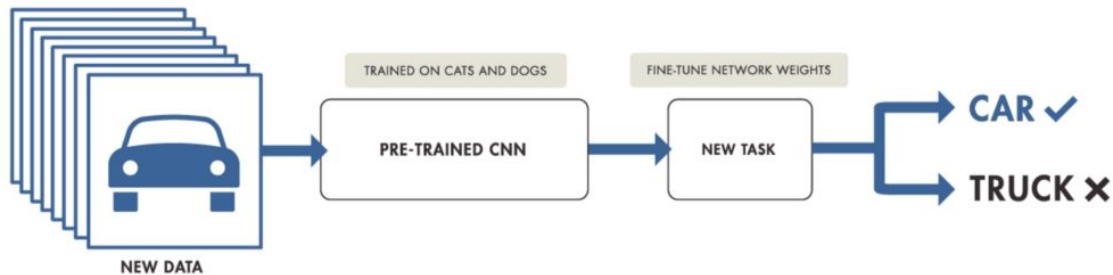
Typically, a combination of several of these methods is used.

Transfer learning

TRAINING FROM SCRATCH



TRANSFER LEARNING



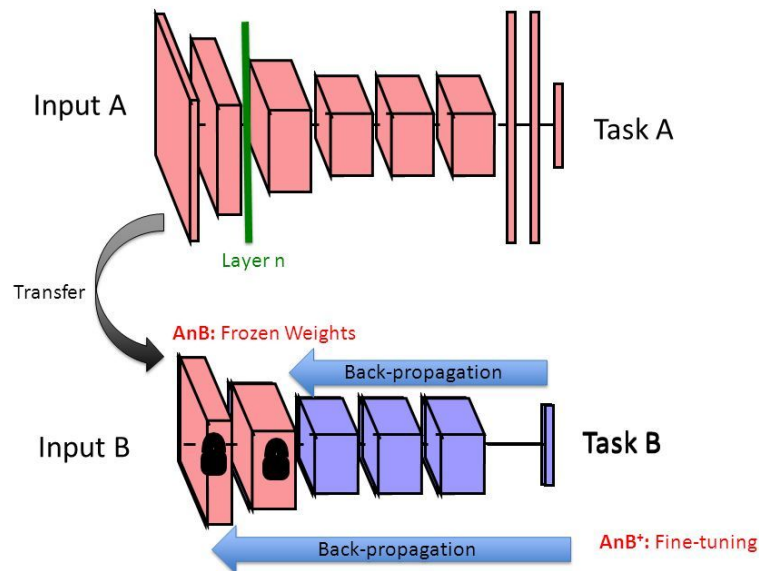
Freezing layers by transfer learning

Freezing layers in a CNN is a technique where some layers are kept fixed during training, meaning their weights are not updated.

This is a common practice in transfer learning to preserve features learned by a pre-trained model, reduce computational cost, and prevent overfitting.

Layers that capture general features are often frozen, while later layers that need to adapt to a specific new task are left unfrozen.

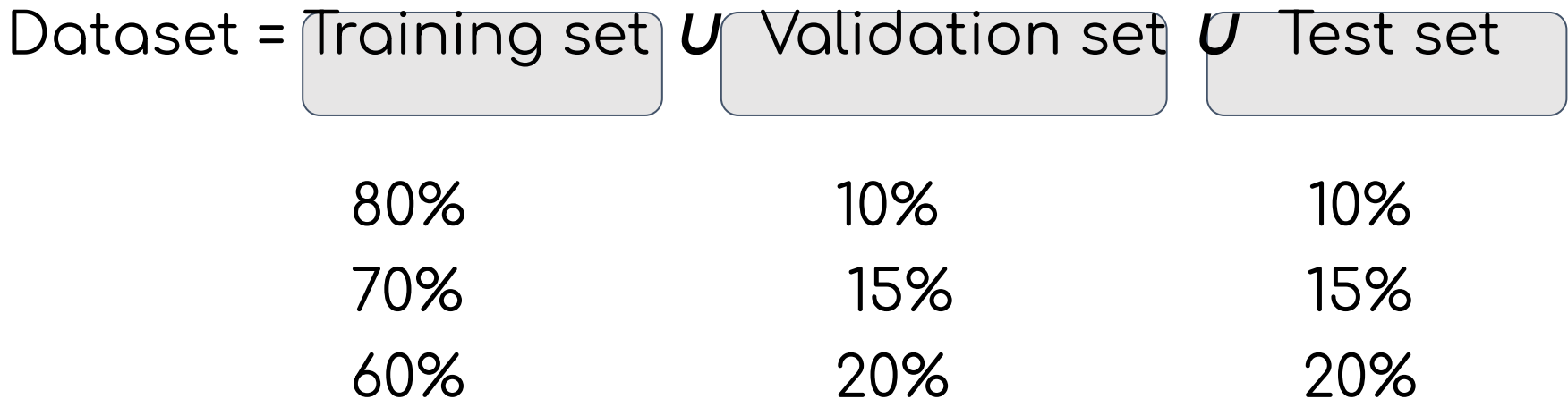
Transfer Learning Overview



Generic ML principles related to training data

Splitting data

A dataset is typically divided into three main subsets:



Splitting data

1. Training Set

used to set the model's parameters (weights and biases).

2. Validation Set

evaluation during the training

helps detect overfitting — when the model performs well on training data but poorly on unseen data.

3. Test Set

used after training is complete to provide an unbiased evaluation of the model's generalization performance.

The test set is never used for training or hyperparameter tuning.

Balancing training data

Balancing training data involves **adjusting the distribution of classes** in a dataset so that each class has a similar number of examples, preventing machine learning models from being biased toward the majority class.

Key techniques include:

- oversampling (duplicating or creating synthetic samples for the minority class),
- undersampling (removing samples from the majority class),
- and using algorithmic approaches like cost-sensitive learning which assigns a higher weight to minority classes during training.

Data augmentation

Data augmentation is a technique used to increase the number of data in the dataset without actually collecting new data.

Basic transformations:

1. Geometric Transformations

Rotation, Flipping, Scaling, Translation, Shearing

2. Color Space Augmentations

Color space augmentations modify the color properties of an image.

3. Filtration, adding Noise

Kernel filters apply convolutional operations to enhance or suppress specific features in the image. It includes: Blurring, Sharpening...

Synthetic Image Generation (Generative Models)

In addition to simple transformations, modern augmentation uses AI-based image synthesis to generate entirely new samples.

1. GAN-based Synthesis (Generative Adversarial Networks)

A GAN consists of a Generator (G) and a Discriminator (D) trained in opposition:

Used in face generation (StyleGAN), medical image synthesis, or domain adaptation (CycleGAN).

Synthetic Image Generation (Generative Models)

2. Diffusion Models

- Iteratively denoise random noise to generate realistic images.

- Capable of producing high-quality, diverse, and controllable synthetic images.

- Often used for augmenting rare categories or generating domain-specific variations.

3. 3D Simulation-Based Synthesis

- Uses rendering engines (e.g., Unity, Blender, CARLA) to generate labeled images under controlled environments.

- Allows systematic variation of lighting, viewpoint, weather, and object poses.

- Common in autonomous driving, robotics, and industrial inspection.

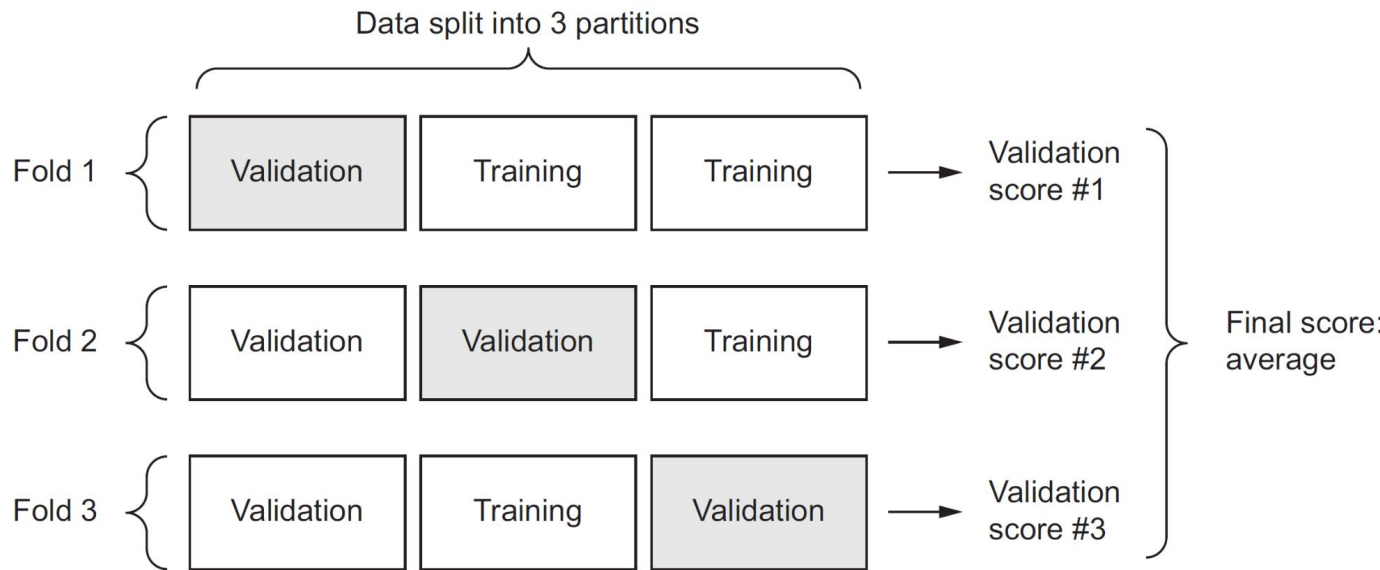
k-fold validation

If you have few data, the validation set would end up being very small. As a consequence, the validation scores might change a lot depending on which data you chose to use for validation and which you chose for training.

This would prevent you from reliably evaluating your model.

The best practice in such situations is to use K-fold cross-validation

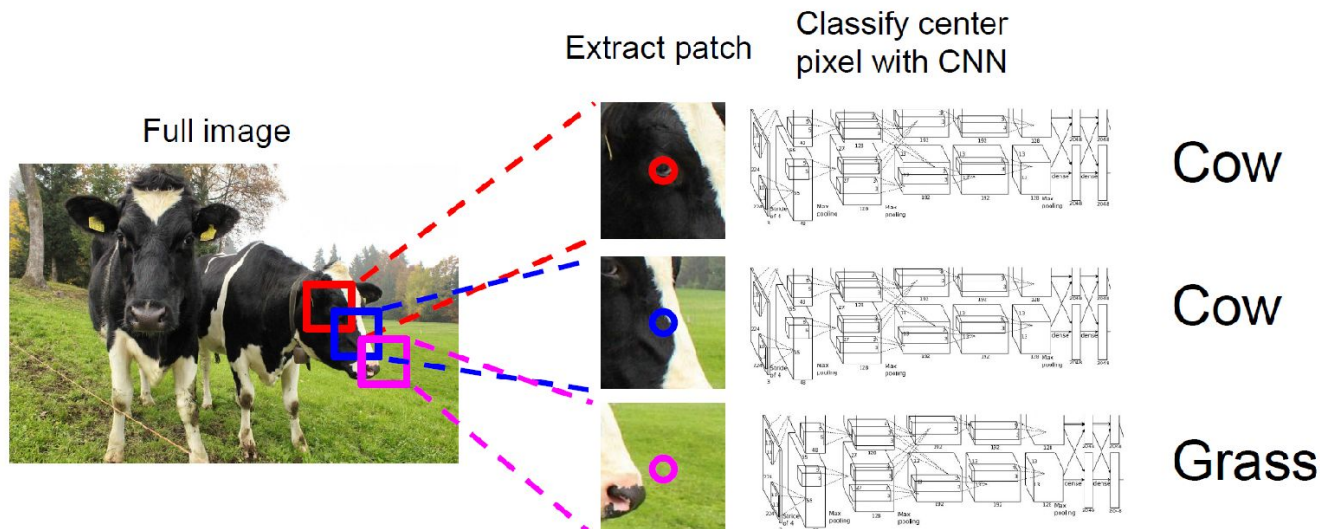
3-fold validation



Object segmentation using CNN

Semantic Segmentation Idea: Sliding Window

Problem: Very inefficient! Not reusing shared features between overlapping patches



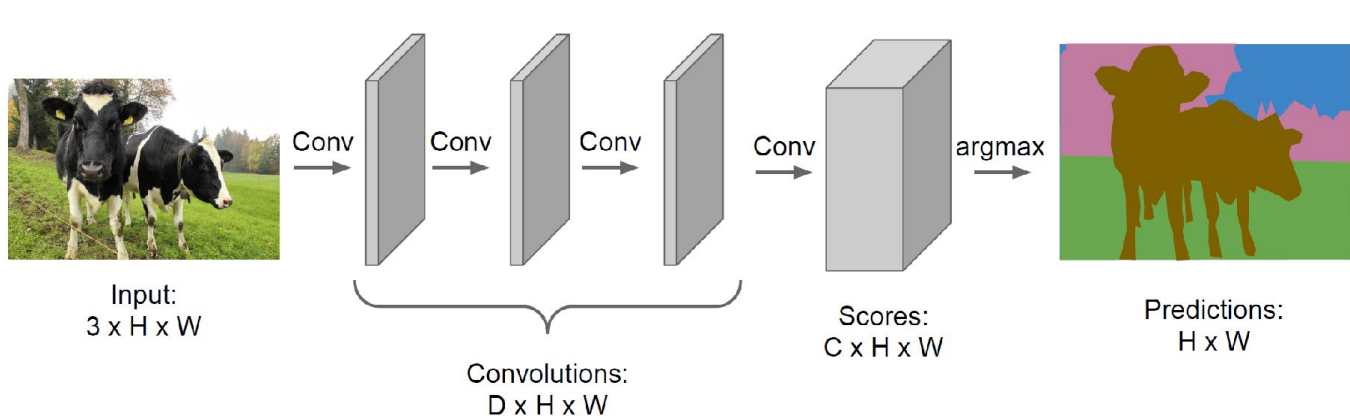
Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013

Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

<http://cs231n.stanford.edu/slides/2019/>

Semantic Segmentation Idea: Fully Convolutional

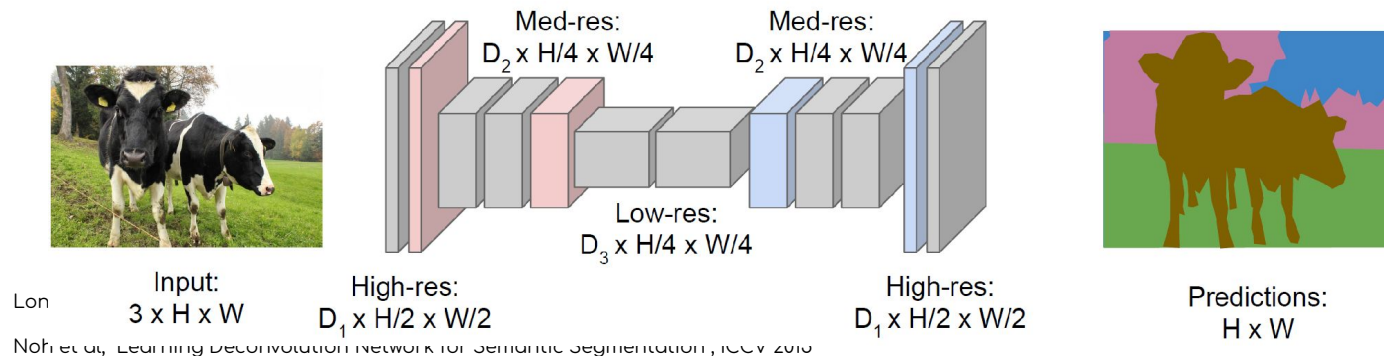
Design a network as a bunch of convolutional layers to make predictions for pixels all at once!



Problem: convolutions at original image resolution will be very expensive ...

Semantic Segmentation Idea: Fully Convolutional

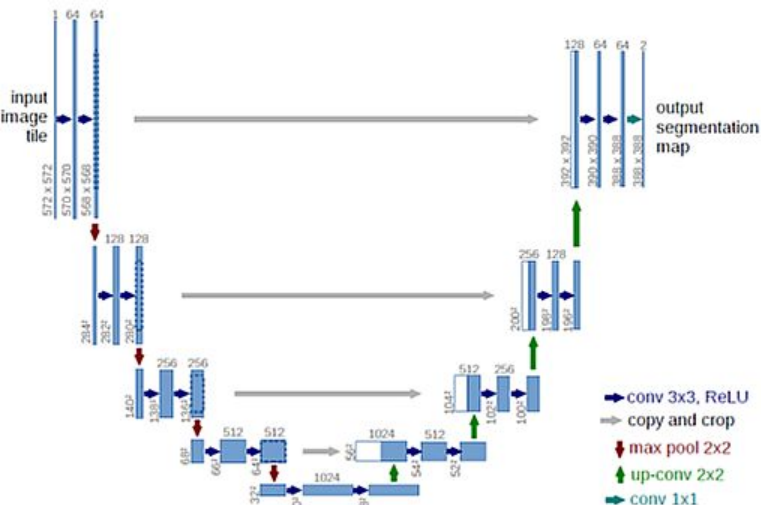
Design network as a bunch of convolutional layers, with downsampling and upsampling inside the network!



Long
Noh et al., Learning deconvolution network for semantic segmentation, ICCV 2015

U-Net architecture

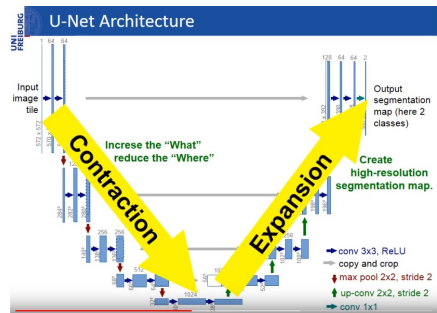
example for 32x32 pixels in the lowest resolution:



- Each blue box corresponds to a multi-channel feature map.
- The number of channels is denoted on top of the box.
- The x-y-size is provided at the lower left edge of the box.
- White boxes represent copied feature maps.
- The arrows denote the different operations.

U-Net architecture

It consists of
a contracting path (left side)
and
an expansive path (right side).



contracting path (left side)

Follows the typical architecture of a convolutional network:

- It consists of the repeated application of two 3x3 convolutions (unpadded convolutions), each followed by a rectified linear unit (ReLU) and a 2x2 max pooling operation with stride 2 for downsampling.
- At each downsampling step we double the number of feature channels.

expansive path (right side)

Every step in the expansive path consists of

- upsampling of the feature map followed by a 2x2 convolution ("up-convolution") that halves the number of feature channels,
- a concatenation with the correspondingly cropped feature map from the contracting path,
- and two 3x3 convolutions, each followed by a ReLU. The cropping is necessary due to the loss of border pixels in every convolution.
- At the final layer a 1x1 convolution is used to map each 64- component feature vector to the desired number of classes.

2D Transposed Convolution

- „Deconvolution block“

The 2D “Deconvolution block” represents a layer that performs an **opposite operation** to 2D Convolution.

The term „Deconvolution“ is not exact – NOT a Deconvolution in terms of signal theory

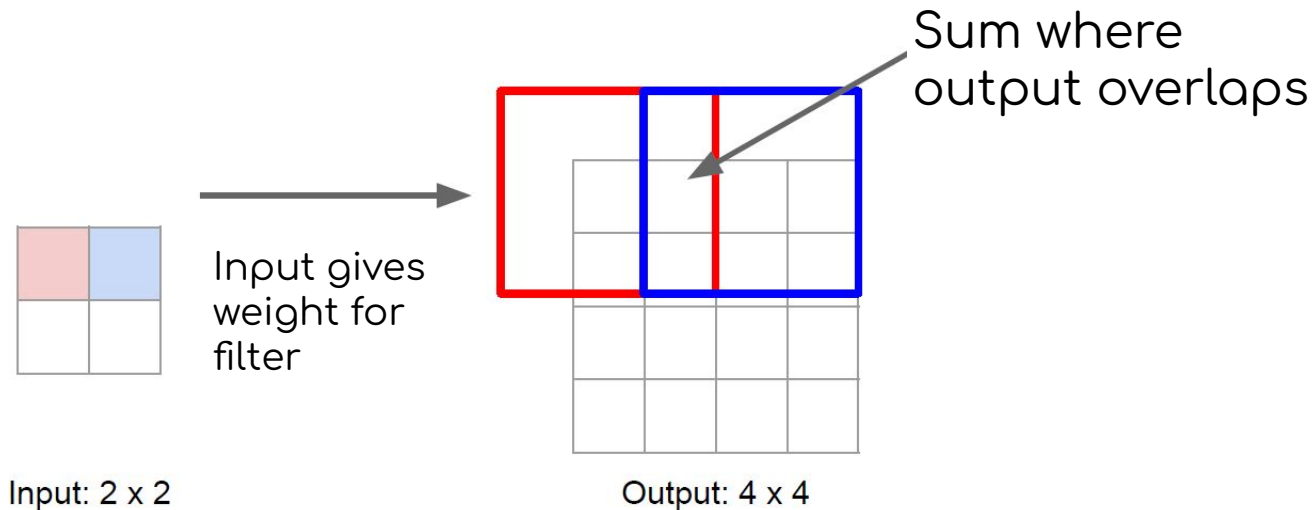
- Deconvolution blocks **have to learn weights** in the same way as Convolution layers
- operate similarly to convolution blocks, **but stride has the opposite effect** (making the output bigger, not smaller).

Usage: The deconvolution operation is used when we transform the output of a convolution back into a tensor that has the same shape as the input of that convolution.

Autoencoders and image segmentation models (U net) are examples of such models.

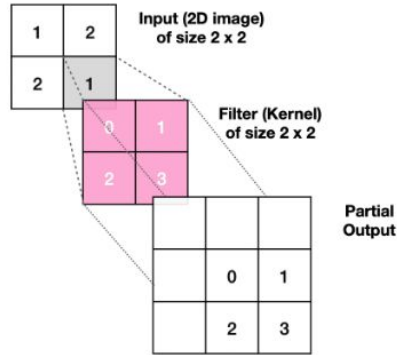
Learnable Upsampling: Transpose Convolution

3 x 3 transpose convolution, stride 2 pad 1



2D Transposed Convolution - example

Transposed
convolution layer
with a 2x2 kernel.



0	1	
2	3	

 +

	0	2
	4	6

 +

0	2	
4	6	

 +

	0	1
	2	3

 =

=

0	1	2
2	9	7
4	8	3

Final
Output

The final output is a 3x3 matrix with values 0, 1, 2, 2, 9, 7, 4, 8, 3.

Transpose Convolution

Other names:

- Deconvolution (not exact)
- Upconvolution
- Fractionally strided convolution
- Backward strided convolution

U-Net Training

The **input images and their corresponding segmentation maps** are used to train the network with the stochastic gradient descent (SGD).

Loss function:

Binary cross-entropy loss — In the context of binary classification in neural networks, the cross-entropy loss $L(z,y)$ is commonly used and is defined as follows:

$$L(z,y) = -[y \cdot \log(z) + (1-y) \cdot \log(1-z)]$$

z : outputs predicted by the model, y : actual outputs

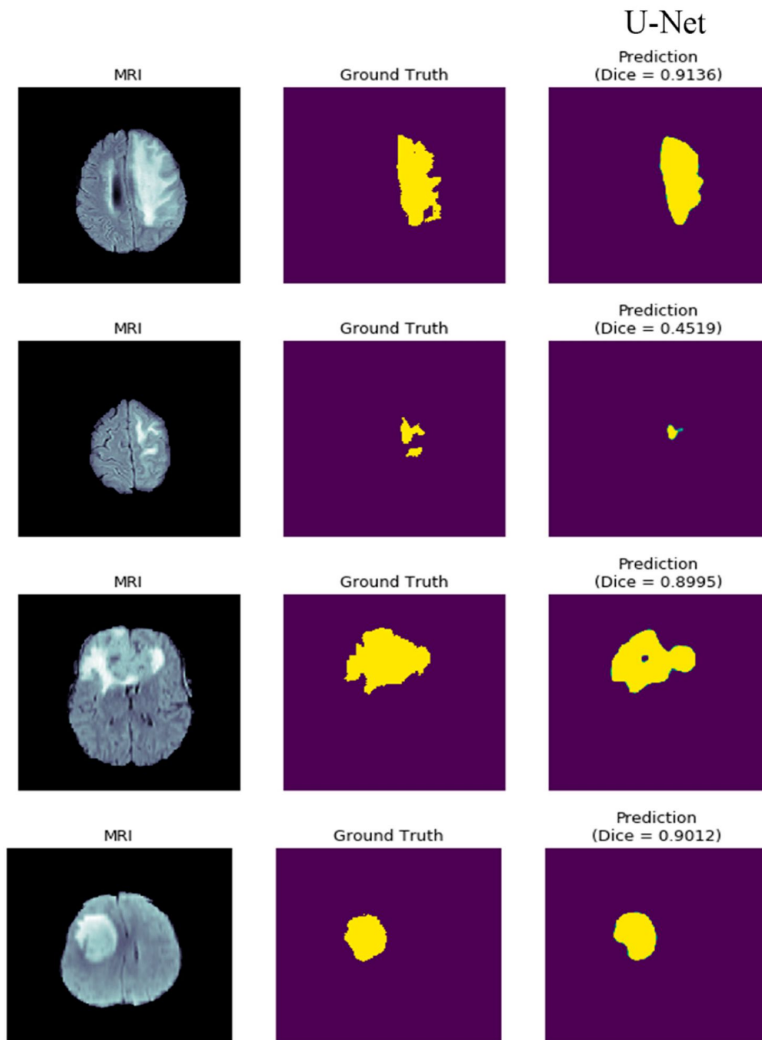
Dice loss:

One minus the Dice index

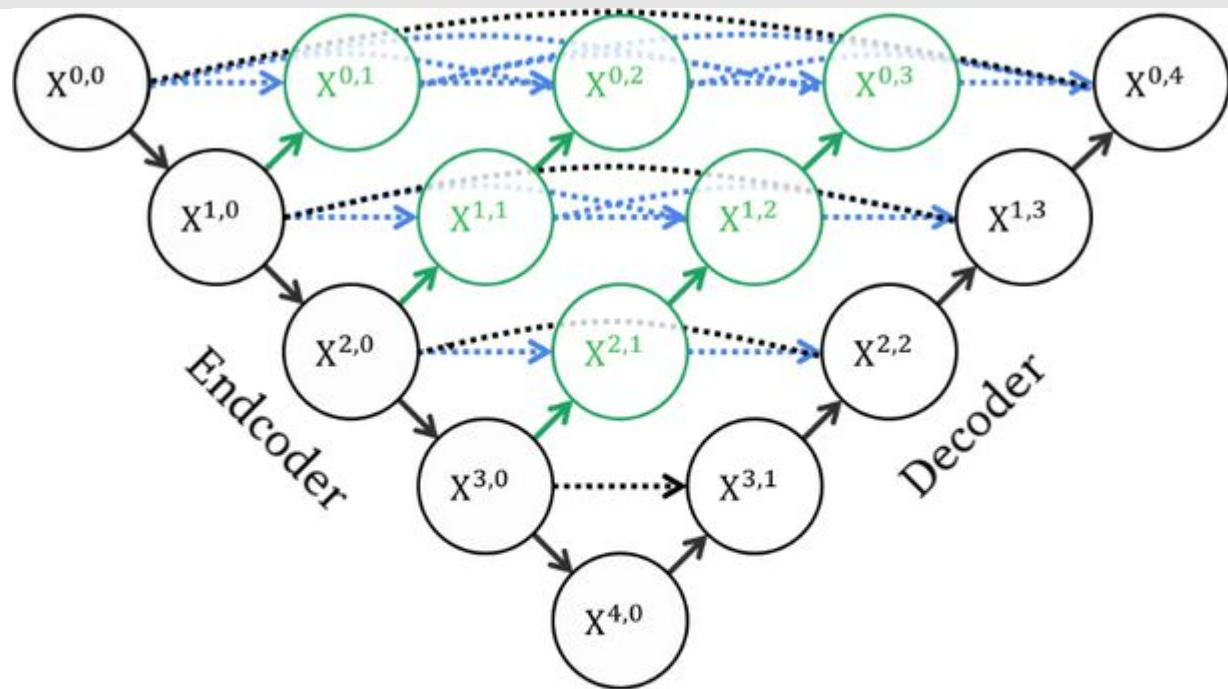
$$DICE = \frac{2 |S_g^1 \cap S_t^1|}{|S_g^1| + |S_t^1|} = \frac{2TP}{2TP + FP + FN}$$

Example:

Brain tumor segmentation using U-net



U-Net ++



$$x^{0,1} = H[x^{0,0}, U(x^{1,0})], \quad x^{0,2} = H[x^{0,0}, x^{0,1}, U(x^{1,1})], \quad \dots$$

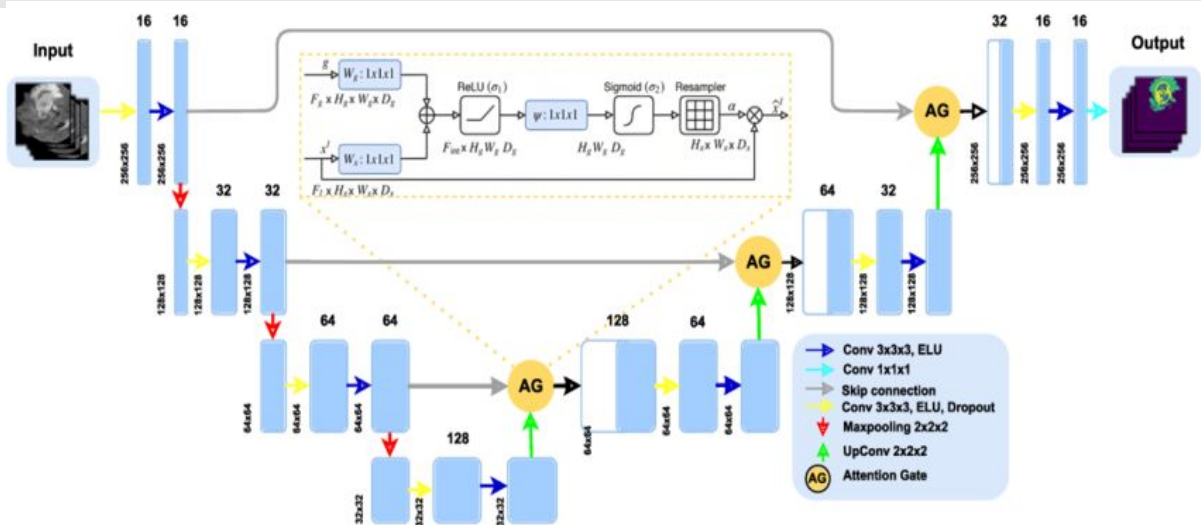


Down-sampling (2*2)



Skip connection

Attention U-net



The attention gate AG involves two input vectors: **X** and **G**.

Vector G is derived from the lower layer of the network and has smaller dimensions but better feature representation.

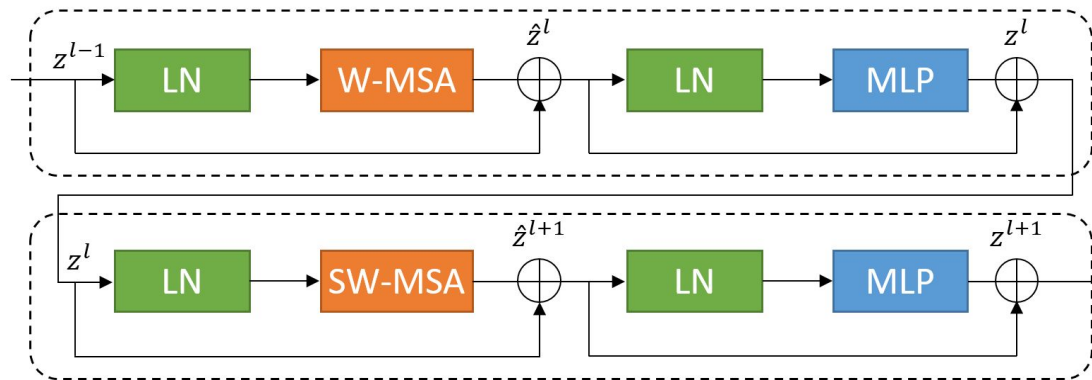
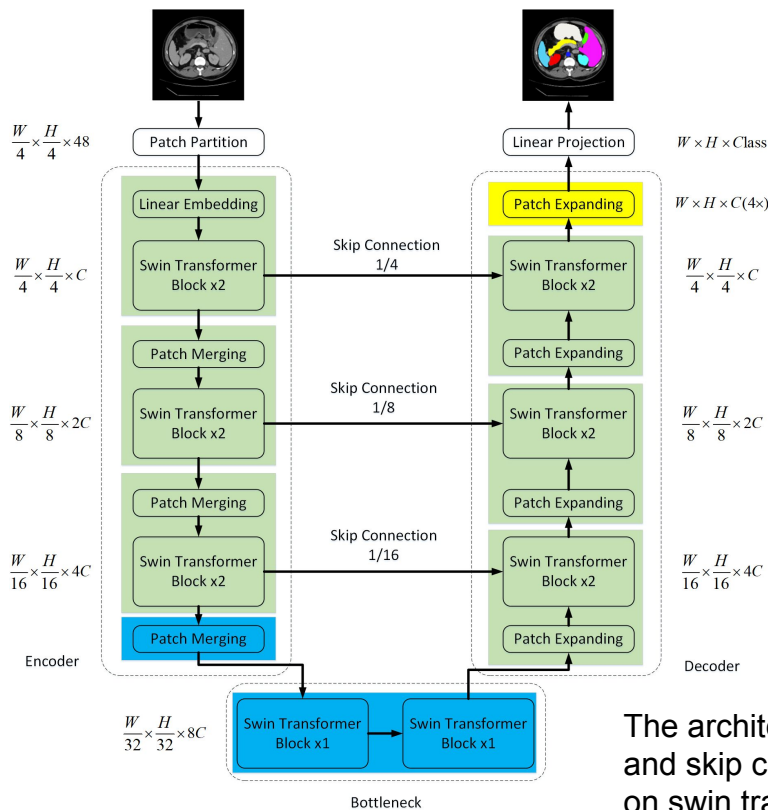
Vector X undergoes a strided convolution, while vector **G** undergoes a 1x1 convolution.

These vectors are then combined through element-wise summation, resulting in a new vector. This new vector is passed through activation and convolutional layers to reduce its dimensions. A sigmoid layer is then applied to produce attention coefficients, which represent the importance of each element in vector **X**.

To restore the attention coefficients to the original dimensions, trilinear interpolation is used. The attention coefficients are multiplied with the original vector **X**, scaling it based on relevance. Finally, the modified vector is passed through the skip connection for further

Transformer based U-net

Swin-Unet



Swin transformer block.
is composed of LayerNorm (LN) layer, multi-head self attention module, residual connection and 2-layer MLP with GELU non-linearity.

The architecture of Swin-U-Net, which is composed of encoder, bottleneck, decoder and skip connections. Encoder, bottleneck and decoder are all constructed based on swin transformer block.